arXiv:2605.19078v1 [cs.DS] 18 May 2026

Near-Resolution of the Tradeoff Conjecture in Distributed Proof Labeling Schemes

Arnold Filtser
Bar-Ilan University
arnold.filtser@biu.ac.il

Orr Fischer
Bar-Ilan University
orr.fischer@biu.ac.il

Abstract

In the t-Proof Labeling Scheme model (t-PLS model), our goal is to certify that a network of nodes satisfies a given property P. A prover assigns a label to each node, and each node decides to accept or reject based on its labeled t-hop neighborhood. If P holds, there exists a labeling that makes all nodes accept. If P does not hold, in all labelings at least one node rejects. The cost of a scheme is its maximum label size.

The *Tradeoff Conjecture* [Feuilloley, Fraigniaud, Hirvonen, Paz, and Perry, DISC 18, Dist. Comput. 21] hypothesizes that the existence of a 1-PLS for a property P with cost p implies the existence of a t-PLS for P with cost $O(\lceil p/t \rceil)$. The conjecture was initially shown to hold for specific graph classes, such as trees, cycles, and grids. Later, a weaker $\widetilde{O}(\lceil \Delta p/\sqrt{t} \rceil)$ cost was shown for fixed minor-free graphs, where Δ is the maximum degree.

In this work we resolve the Tradeoff Conjecture, up to a single logarithmic factor. In general graphs, we show that the existence of a 1-PLS with cost p implies the existence of an $O(t \log n)$-PLS with cost $O(\lceil p/t \rceil)$ for the same property. For fixed minor-free graphs (which include e.g. planar graphs), we show that the existence of a 1-PLS with cost p implies the existence of a t-PLS with cost $O(\lceil p/t \rceil + \log n)$ for the same property. We also refute a previously suggested stronger variant of the Tradeoff Conjecture, and show that having very large t-hop neighborhoods is an insufficient condition for obtaining a tradeoff better than $O(\lceil p/t \rceil)$.

Contents

1 Introduction **1**
1.1 Technical Overview 2

2 Preliminaries **8**

3 Two-Separated Partitions and Cluster-Degeneracy **10**

4 Tradeoff Theorem for Minor-Free Graphs **16**

5 Tradeoff Theorem for General Graphs **18**
5.1 Helpful Sequential Algorithm for Finding a $(O(t \log n), 1/t)$-TS Partition . . 18
5.2 Labeling Scheme for General Graphs . 21

6 Refuting Stronger Variants of the Tradeoff Conjecture **25**

A Missing Proofs from Section 4 **30**

B Missing Proofs from Section 5 **32**

1 Introduction

In distributed proof labeling schemes (the PLS model), our goal is to verify that a network of nodes satisfies a given property P. The PLS model was first introduced by [KKP10], and has been studied in the context of distributed verification, with the aim of locally certifying that a network is in a valid state while storing compact labels on the nodes. Beyond verification, the PLS model has direct applications in the design of self-stabilizing protocols and in the study of local algorithms (See [Feu21] for a survey).

Formally, for a graph $G = (V, E)$ with some auxiliary inputs I (e.g. edge weights), a proof labeling scheme certifying a property P of (G, I) consists of a pair $\Pi = (\text{Prov}, \text{Ver})$, where Prov is a prover strategy that assigns a label to each node, and Ver is a verification algorithm executed by every node following a label assignment. Each node v decides to accept or reject based on its label and the label of its neighbors. The scheme must satisfy two conditions:

- *Completeness:* if (G, I) satisfies P, all nodes accept given the labeling of the prover Prov.
- *Soundness:* if (G, I) does not satisfy P, then for every labeling at least one node rejects.

The cost of a scheme Π is the maximum label size assigned by the prover Prov, as a function of the graph size $|V(G)| = n$.

In the generalized t-PLS model, nodes may decide to accept or reject based on their t-hop neighborhood and the labels of the nodes in this neighborhood, instead of just their immediate neighborhood. This extension is motivated by the study of space-time tradeoffs, as it allows the verifier to use t rounds of communication (instead of one) to potentially reduce the label sizes. In this model, many tradeoffs have been studied for specific problems, such as acyclicity [OPR17], MST and shortest path problems [FFH+21], colorings [ACF+23, FJKS24], and dominating sets [FJKS24]. On the other hand, some works investigated space-time tradeoffs of any property P [OPR17, FFH+21, FOS21, BFZ25]. The focus of our work is on this latter category.

The t-PLS model was first defined by Ostrovsky, Perry, and Rosenbaum [OPR17], who showed that any property P can be certified by a t-PLS with cost $O(\min(n^2, m \log n)/t)$. Following this work, Feuilloley, Fraigniaud, Hirvonen, Paz, and Perry [FFH+21] showed that for trees, cycles, and grids, any 1-PLS for a predicate P with cost p can be transformed into a t-PLS for P with cost $O(\lceil p/t \rceil)$. They raised the question of whether this tradeoff can be obtained in general, in a conjecture later known as the Tradeoff Conjecture. The conjecture was raised again as an open question in [FOS21, ACF+23, FJKS24, BFZ25] and the survey [Feu21].

Conjecture 1.1 (The Tradeoff Conjecture [FFH+21])**.** *If there exists a 1-PLS for a predicate P on all graphs with cost $p = p(n)$, then for any $t \geq 1$, there exists a t-PLS for P with cost $O(\lceil p/t \rceil)$.*

We resolve the weak version of the conjecture (mentioned as Problem 1 in [FFH+21]). Specifically, we prove that the Tradeoff Conjecture holds, up to a single *multiplicative* $\log n$ factor. To our knowledge, this is the first tradeoff for any P and for all graphs depending on the cost of the 1-PLS. In the following, a configuration family $(\mathcal{G}, \mathcal{I})$ is a pair consisting of a graph family $\mathcal{G}$ and a set of possible auxiliary inputs $\mathcal{I}$.

Theorem 1.2. *Let $(\mathcal{G}, \mathcal{I})$ be a configuration family, and P a predicate on $(\mathcal{G}, \mathcal{I})$. For $G \in \mathcal{G}$ with $|V(G)| = n$, if there exists a 1-PLS for P with cost p, then for any $t \geq 1$ there exists an $O(t \log n)$-PLS for P with cost $O(\lceil p/t \rceil)$.*

Consider the problem of MST verification in graphs with integer polynomial edge weights. Kutten and Peleg [KK07] constructed a 1-PLS with cost $O(\log^2 n)$. Later, Feuilloley *et al.* [FFH+21] constructed a t-PLS with cost $O(\lceil \log^2 n/t \rceil)$. However, their t-PLS can only be applied for $t = O(\log n)$. Thus, the scheme of [FFH+21] has cost $\Omega(\log n)$ even for very large t. By applying our black-box Theorem 1.2 combined with [KK07], we get a general tradeoff, and an improvement over [FFH+21] for the parameter range $t \geq \log^2 n$. In particular, for $t = \Omega(\log^3 n)$, we get constant cost.

Corollary 1.3 (t-PLS for MST verification)**.** *For any $t \geq \log n$, there exists a t-PLS for the MST verification problem with cost $O(\lceil \log^3 n/t \rceil)$.*

Next, we prove that the Tradeoff Conjecture holds in fixed minor-free graphs, up to a single *additive* $\log n$ factor. Previously, Fischer, Oshman and Shamir [FOS21] showed a bound of $\widetilde{O}(\lceil \Delta p/\sqrt{t} \rceil)$ on the cost, where Δ is the maximum degree in the graph. Our result removes the dependence in Δ, and improves the dependence on t quadratically. Many well-known graph families are minor free, such as planar graphs, as well as graphs of bounded genus or treewidth. We state our results for K_r-minor free graphs, which are in particular H-minor free for any H such that $|V(H)| \leq r$.

Theorem 1.4. *Let $r \geq 1$ be a constant, and let $(\mathcal{G}, \mathcal{I})$ be a configuration family where $\mathcal{G}$ is the family of all K_r-minor free graphs, and let P be a predicate on $(\mathcal{G}, \mathcal{I})$. If there exists a 1-PLS for P in $(\mathcal{G}, \mathcal{I})$ with cost p, then for any $t \geq 1$, there exists a t-PLS for P in $(\mathcal{G}, \mathcal{I})$ with cost $O(\lceil p/t \rceil + \log n)$.*

In [FFH+21], the authors additionally asked whether the tradeoff might scale linearly with the ball growth of the graph $b(t) = \min_v |B_t(v)|$,[1] although they considered this to be most likely false.

Question 1.5 ([FFH+21] Open Problem 2)**.** *Assuming there exists a 1-PLS for a predicate P on graphs with cost $p = p(n)$, does there exist for any $t \geq 1$ a t-PLS for P with cost $\widetilde{O}(\lceil p/b(t) \rceil)$?*

We confirm that this does not hold, and show that even if all t-neighborhoods are of size $\Omega(n)$, the best scaling possible is $\Omega(\lceil p/t \rceil)$. Moreover, this phenomenon already occurs in planar graphs.

Theorem 1.6. *There exists a predicate P on an infinite family of planar graphs $\mathcal{G} = \{G_{t,m}\}_{t,m=1}^{\infty}$ and on some auxiliary input set $\mathcal{I}$, where for $G_{t,m} \in \mathcal{G}$ we have $\min_{v \in V(G_{t,m})} |B_t(v)| = \Omega(|V(G_{t,m})|)$, and there is a 1-PLS for P with cost $O(m)$, but any t-PLS for P has cost $\Omega(m/t)$.*

Additional related works: The PLS model (i.e. 1-PLS model) has been extensively studied in a broad range of topics, such as for various graph properties [KK07, KKP10, GS16, CHPP20], for properties expressible in some logic frameworks [FBP22, FMRT24, CKM25], and for memberships in graph classes [FFM+21, BFP24]. Given the very rich literature on PLS, we refer to the survey on local certification and the PLS model of Feuilloley [Feu21], as well as the surveys on related topics of Feuilloley and Fraigniaud [FF16], and of Suomela [Suo13].

1.1 Technical Overview

We begin by presenting a key framework for obtaining tradeoff theorems, used both in prior works and in our work, which shows how the existence of certain graph decompositions directly yields

[1] $B_t(v) = \{u \in V \mid \mathrm{dist}_G(v,u) \leq t\}$ denotes the ball around v of radius t w.r.t. the shortest path metric of G.

tradeoff theorems. We then introduce a new type of decomposition, called *Two-Separated partitions* (TS partitions), which extends the prior decompositions while implying similar tradeoff theorems.

Following this, we show three constructions of TS partitions in graphs: (a) a warmup construction which yields a weaker tradeoff theorem for general graphs (cf. Theorem 1.2), but its ideas are used throughout the paper, (b) constructing TS partitions from padded decompositions (introduced later), which implies our results for minor free graphs (Theorem 1.4), and (c) a construction which yields our tradeoff theorem for general graphs (Theorem 1.2). Finally, we discuss the techniques for refuting the stronger variant of the Tradeoff Conjecture (Theorem 1.6).

Tradeoff from partition-based decompositions: To obtain Theorems 1.2 and 1.4, we use a variation on a partition-based framework previously used in [FFH+21, FOS21]. Before introducing our variation, we first informally describe this framework as it appears in prior works.

The key combinatorial object needed for this framework is the existence of a decomposition of the graph into connected clusters $C_1, \ldots, C_k$, each of diameter at most $\alpha \cdot t$ (for some small enough constant $\alpha \leq 1$), such that for the set of boundary vertices[2] X_i of C_i we have that $\forall_i |X_i|/|C_i| \leq \varepsilon$ for some small $\varepsilon = \varepsilon(n,t)$. Given such a decomposition, we describe how to construct a t-PLS Π_t from a 1-PLS Π_1, while paying only roughly an ε-fraction of the cost of Π_1. Specifically, if Π_1 costs p, we need to construct a labeling for every graph G satisfying P, such that the maximum label size is roughly $O(\lceil \varepsilon p \rceil)$, and describe a verifier function that accepts this labeling, but rejects all labelings on graphs not satisfying P.

Label construction for Π_t: Recall that G satisfies P if and only if there is some labeling ℓ_V for Π_1, such that all nodes accept according to Π_1. We describe a t-PLS Π_t that certifies P by proving to the nodes that such a labeling ℓ_V exists. Assume that a 1-PLS ℓ_V exists with cost p. We create the t-PLS labels of the vertices in cluster C_i by taking the 1-PLS labels of its boundary vertices $\{\ell_V(x)\}_{x \in X_i}$, and splitting them among all the cluster vertices (say lexicographically). Additionally, each vertex $v \in C_i$ will get an *auxiliary label* containing additional information so that v can identify its cluster C_i. We denote the maximum possible size of the auxiliary label by c, and call it *description cost*. The resulting label size of C_i vertices will be $O\left(\left\lceil \frac{|X_i|}{|C_i|} \cdot p \right\rceil + c\right) = O\left(\lceil \varepsilon p \rceil + c\right)$. Thus the cost of Π_t is $O\left(\lceil \varepsilon p \rceil + c\right)$.

Verification under Π_t: For each cluster C_i, a leader $v_i \in C_i$ is designated. Given the labels of C_i, v_i can reconstruct the labels $\ell_V(x)$ for all boundary vertices $x \in X_i$. Further, let $\bar{C}_i$ be the collection of clusters with edges also incident to C_i (i.e. neighboring clusters, excluding C_i), and by $\bar{X}_i = \cup_{C_j \in \bar{C}_i} X_j$ the union of their boundary vertices. Note that for each cluster $C_j \in \bar{C}_i$, as all C_j vertices are at distance $\leq 2\alpha \cdot t$ from v_i, v_i can reconstruct the cluster C_j, and the Π_1-labels of its boundary vertices X_j. In other words, v_i can compute $\{\ell_V(x)\}_{x \in \bar{X}_i}$. We observe the following:

- For every $v \in C_i \setminus X_i$, all its neighbors are in C_i.
- For every $x \in X_i$, all its neighbors are in $C_i \cup \bar{X}_i$.

All C_i vertices other than the leader v_i will accept regardless of the input. v_i will accept iff there are labels $\{\ell_V(v)\}_{v \in C_i \setminus X_i}$ for the non-boundary vertices so that all C_i vertices (including X_i) will be accepted by the Π_1-verifier simultaneously (with accordance to the given labels of $\bar{X}_i \cup X_i$).

[2]The boundary X_i of a cluster C_i consists of all the vertices in C_i with neighbors out of C_i. Formally, $X_i = C_i \cap B_1(V \setminus C_i)$, where $B_1(S)$ is the ball of radius 1 around S.

Showing completeness and soundness is now straightforward. If (G, I) satisfies P, and a Π_1 labels indeed exists, each leader v_i can use them to convince itself that there are satisfying labels and thus will accept. For soundness, one can use the fact that there are no edges between yet undetermined (i.e. non-boundary) vertices in different clusters, and thus if each leader v_i accepts separately, the combined labels they computed will satisfy the Π_1-verifier, and thus by the soundness of Π_1, (G, I) satisfies P.

Challenge: As it turns out, constructing decompositions such that the fraction of boundary vertices is small in all the clusters simultaneously is quite challenging. In the following, we describe how we can relax the requirements from the decomposition to get considerably better results.

New combinatorial notions - Two-Separated partitions and cluster-degeneracy: In the previous part, we discussed how the existence of a decomposition into connected clusters $C_1, \ldots, C_k$, with diameter $O(t)$ and boundary-to-size ratio of $|X_i|/|C_i| \leq \varepsilon$ directly implies a tradeoff theorem in which the t-PLS costs an ε-fraction of the original 1-PLS. As our first contribution, we present the notion of a Two-Separated partition (TS partition), which generalizes the decomposition above while still implying the same tradeoff guarantees.

A (t, ε)-*Two-Separated partition* is a pair $(\mathcal{C}, X)$ where $X \subseteq V$ is called the *separating set*, and $\mathcal{C}$ is a partition of V, satisfying (a) for $i \neq j$, any path between two nodes $u \in C_i \setminus X$ and $v \in C_j \setminus X$ must contain two consecutive vertices in X, (b) every cluster in $\mathcal{C}$ has weak diameter at most t,[3] and (c) $\max_i |C_i \cap X|/|C_i| \leq \varepsilon$. (See Figure 1(a) for an example)

Conceptually, this new decomposition allows the set $X_i = C_i \cap X$ to differ from the boundary of the cluster C_i. This allows, for example, for a cluster to be disconnected, as long as it has a low weak-diameter, and also allows a more flexible manner to balance how much "boundary information" is encoded on each cluster. Using essentially the same technique as the partition-based schemes, we show that the existence of a (t, ε)-TS partition implies that a 1-PLS with cost p can be turned into an "equivalent" $O(t)$-PLS with cost $O(\lceil \varepsilon p \rceil + c)$ for some description cost c.

Our main method in the paper of constructing (t, ε)-TS partitions is through the notion of ε-cluster-degeneracy, introduced next. A partition of V into an ordered set of clusters $\mathcal{C} = \{C_1, \ldots, C_k\}$ has ε-cluster-degeneracy if

$$\forall i \in [k], \quad \left|\left\{v \in C_i \mid \mathrm{dist}_G\left(v, V \setminus \cup_{j=1}^{i} C_j\right) \leq 2\right\}\right| \leq \varepsilon \cdot |C_i|.$$

See Figure 1(b) for a pictorial example. We show that a decomposition into clusters with ε-cluster-degeneracy and weak diameter at most t implies the existence of a (t, ε)-TS partition (Lemma 3.1). As stated above, the existence of these decompositions directly implies a tradeoff theorem, where the cost is $O(\lceil \varepsilon p \rceil)$ + the description cost of the TS partition. Thus our primary effort is aimed at showing the existence of TS partitions with small description cost. In the following parts we present three constructions of TS partitions, for both general and minor free graphs.

Warmup - a simple solution for general graphs: As a warmup, we consider a simple ball carving process that outputs for any graph a decomposition into clusters with $(1/t)$-cluster-degeneracy and weak diameter $O(t \log n)$. That is, we show that V can be partitioned into clusters

[3] The weak diameter of a cluster C is $\max_{u,v \in C} \mathrm{dist}_G(u, v)$ the maximum pairwise distance w.r.t. the original distances. In particular, a cluster may be disconnected.

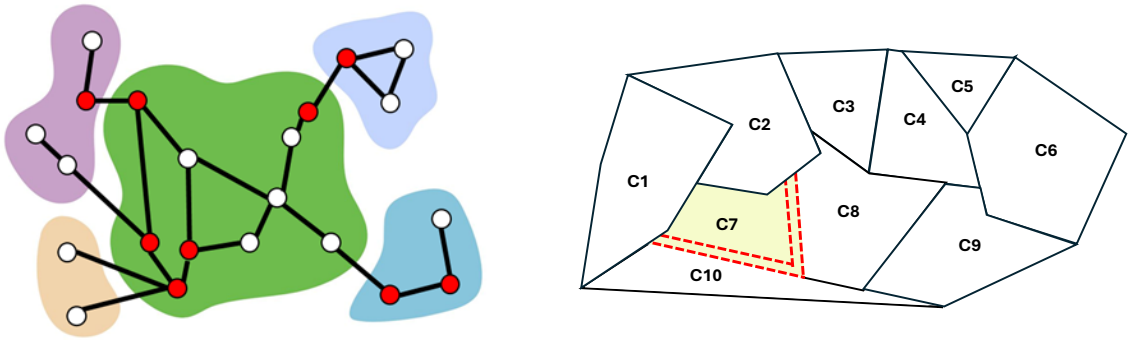


Figure 1: (a) Illustration of a $(5, 2/3)$-TS partition. Nodes of X marked in red. All clusters have weak diameter ≤ 5, where some clusters are internally disconnected. Worst red nodes to size ratio is in the bottom-right cluster, with $\varepsilon = 2/3$. All white-to-white paths between clusters must pass two consecutive red nodes. (b) Illustration of cluster-degeneracy. In the example, we highlight for C_7 its 2-boundary with the clusters that follow it. We remark that in general, clusters can be disconnected.

$C_1, \ldots, C_k$, each of weak diameter $O(t \log n)$, such that

$$\forall i \in [k], \quad \left|\left\{v \in C_i \mid \mathrm{dist}_G\left(v, V \setminus \cup_{j=1}^{i} C_j\right) \leq 2\right\}\right| \leq |C_i|/t.$$

The process works in iterations. For iteration $i \geq 1$, assume we are given some set of alive nodes $L_i \neq \emptyset$, initialized as $L_1 = V$. We take an arbitrary node $v_i \in L_i$, and consider balls of growing radius centered at v_i. We say that a ball $B_j(v_i)$ of radius $j \geq 2$ is *expanding* if $|B_j(v_i) \cap L_i| / |B_{j-2}(v_i) \cap L_i| > 1 + 1/t$. We note that there exists a non-expanding ball centered at v_i of radius $O(t \log n)$. Otherwise, due to the many expanding layers, we have $|B_{O(t \log n)}(v_i)| \geq (1 + 1/t)^{\Theta(t \log n)} > n$.[4] If we find a non-expanding ball $B_j(v_i)$, we define the cluster $C_i = B_j(v_i)$ and set $L_{i+1} = L_i \setminus C_i$, terminating when $L_{i+1} = \emptyset$.

Using simple calculations, it is easy to see that the resulting clusters are $(1/t)$-cluster-degenerate: Each cluster is a non-expanding ball, whose cluster's size is $|B_r(v_i) \cap L_i|$, and its 2-hop boundary with subsequent clusters is $|(B_r(v_i) \setminus B_{r-2}(v_i)) \cap L_i|$. Therefore by definition of a non-expanding radius, we have

$$|(B_r(v_i) \setminus B_{r-2}(v_i)) \cap L_i| = |B_r(v_i) \cap L_i| - |B_{r-2}(v_i) \cap L_i| \leq (1/t) \cdot |B_r(v_i) \cap L_i| = |C_i|/t \ .$$

We conclude that a $(O(t \log n), 1/t)$-TS partition exists in any graph. Furthermore, using standard arguments, one can show that any TS partition can be described with a cost of $O(\log n)$ bits (See Lemma 4.5). This immediately implies that any 1-PLS for a property P with cost p can be transformed to an $O(t \log n)$-PLS for P with cost $O(\lceil p/t \rceil + \log n)$.

However, many properties of interest (e.g. computing a spanning tree [KKP10, GS16], MST [KK07], or certifying planarity [FFM+21]) have a 1-PLS with logarithmic or close to logarithmic cost. Ideally we would like a t-PLS for any property, that converges to constant cost as t grows. We could obtain this if we could reduce the additive factor in the tradeoff to be bounded by a constant. Therefore, a natural question to ask is whether there exists a TS partition with the parameters

[4] We remark that if $B_j(v_i)$ contains all alive nodes, then $B_{j+2}(v_i)$ is a non-expanding ball.

above, but with constant description cost? Another natural question is whether we can remove the multiplicative dependency of the tradeoff in n, at least for some large graph class? We positively answer the two questions, as discussed next.

K_r-Minor free graphs and padded decompositions: In Section 4, we show that for the class of K_r-minor free graphs (for constant $r \geq 1$), any 1-PLS for a property P with cost p can be transformed into a t-PLS for P with cost $O(\lceil p/t \rceil + \log n)$, which is optimal up to a single additive logarithmic factor.[5] The main tool we use is the existence of good "padded decompositions" in these graphs.

See Definition 6 for a formal definition of padded decomposition. Here it will suffice to note that if a graph has padding parameter $\beta > 0$, then for any $t = \Omega(1)$, there is a *randomized* partition of V into clusters, such that (a) each cluster has weak diameter at most t, and (b) for every node $v \in V$, $\Pr(B_2(v) \not\subseteq C_v) \leq 2\beta/t$, where C_v is the unique cluster containing v, and $B_2(v)$ is the ball of radius 2 centered at v.[6] The padding parameter β of different graphs is a widely studied topic that has many algorithmic applications (See [CF25] and the references therein). It is known that for K_r-minor free graphs $\beta = \Theta(\log r)$ [CF25] ($O(1)$ if r is constant), while in general graphs $\beta = \Theta(\log n)$ [Bar96].

As we next show, the existence of padded decompositions with low padding parameter β in a graph G implies the existence of a good TS partition in G. We start by showing that graphs with padding parameter β have some cluster $C \subseteq V$ with weak diameter at most t, and $|C \cap B_2(V \setminus C)| \leq (2\beta/t)|C|$. Let us sample a random padded decomposition with padding parameter β and weak diameter t, i.e. a randomized partition into clusters $\{C_1, \ldots, C_k\}$, such that the weak diameter of the clusters is at most t, and that property (b) holds. Let Γ_2 be the 2-boundary of the clusters, i.e. $\Gamma_2 = \{v \mid \mathrm{dist}_G(v, V \setminus C_v) \leq 2\}$. We notice that property (b) of the decomposition is equivalent to:

$$\Pr(v \in \Gamma_2) = \Pr(B_2(v) \not\subseteq C_v) \leq 2\beta/t.$$

Therefore, by linearity of expectation, $\mathbb{E}(|\Gamma_2|) \leq (2\beta/t) \cdot n$. We notice that

$$\min_{i \in [k]} |C_i \cap \Gamma_2|/|C_i| \leq \left(\sum_{i=1}^{k} |C_i \cap \Gamma_2|\right) / \left(\sum_{i=1}^{k} |C_i|\right) = |\Gamma_2|/n,$$

where the first inequality follows from the mediant inequality.[7] In expectation, we have

$$\mathbb{E}(\min_{i \in [k]} |C_i \cap \Gamma_2|/|C_i|) \leq \mathbb{E}(|\Gamma_2|)/n \leq 2\beta/t.$$

We conclude that by the probabilistic method, there exists a cluster C such that $|C \cap B_2(V \setminus C)| \leq (2\beta/t)|C|$. Moreover, using the same argument one can show that for any set $L \neq \emptyset$, we can find a cluster $C \subseteq L$ with weak diameter at most t and $|C \cap B_2(L \setminus C)| \leq (2\beta/t)|C \cap L|$.

Given the existence of such a cluster C, we can perform a similar ball carving process as in the warmup. We define a process with iterations where in iteration $i \geq 1$ we are given some set of

[5] See e.g. Theorem 1.6 for an $\Omega(1/t)$ lower bound on the tradeoff of some problems, though there are many such bounds in the literature.

[6] Recall that the partition is randomized, meaning the randomness is taken over its distribution.

[7] The mediant inequality states that if $a_1, b_1, \ldots, a_\ell, b_\ell > 0$ are real numbers, then $\min_{i \in [\ell]} \frac{a_i}{b_i} \leq \frac{\sum_{i=1}^{\ell} a_i}{\sum_{i=1}^{\ell} b_i}$.

alive nodes $L_i \neq \emptyset$, initialized as $L_1 = V$. Using the padded decomposition, we argue that there exists some cluster C_i such that $|C_i \cap B_2(L \setminus C_i)| \leq (2\beta/t)|C_i|$. We add C_i to our partition, and set $L_{i+1} = L_i \setminus C_i$, terminating if $L_{i+1} = \emptyset$. The resulting partition can be shown to be a partition into clusters with weak diameter t, and $(2\beta/t)$-cluster-degeneracy, which for K_r-minor free graphs translates to $O(1/t)$-cluster-degeneracy, and implies Theorem 1.4.

Interestingly, by using the optimal padded decomposition of general graphs [Bar96], we also obtain an alternative method to the same result as in the warmup - that a 1-PLS for a property P with cost p can be transformed to an $O(t \log n)$-PLS for P with cost $O(\lceil p/t \rceil + \log n)$.

General Graphs, with Constant Description Cost: In Section 5, we show that for the class of all graphs, any 1-PLS for a property P with cost p can be transformed into an $O(t \log n)$-PLS with cost $O(\lceil p/t \rceil)$. This cost's additive factor is bounded by a constant, hence this scheme converges to a constant number of bits as t grows. Our goal is to adapt the construction of the warmup to obtain an $(O(t \log n), 1/t)$-TS partition that can be locally described using $O(1)$ bits per node.

Attempt 1 - Ball carving with advice: We design a compact description for the warmup construction. Recall that in this process we took in each step some living node v, carved a ball $B_r(v)$ around it for some r, added it to our clusters, and removed $B_r(v)$ from the set of living nodes. Let us further assume that we iterate over the vertices in increasing order of id, i.e. in each step we take the node with the smallest identifier that is still alive.

We describe a label of size $O(\log t + \log \log n)$ which allows nodes to recognize their cluster, given their labeled $O(t \log n)$-neighborhood: we mark for each node v if a ball $B_{r(v)}(v)$ was taken as a cluster, and if so, its radius $r(v)$. We note that a node u can locally simulate the process, and deduce its cluster, by finding the node v with the smallest id in its $O(t \log n)$ neighborhood such that $u \in B_{r(v)}(v)$, and assigning itself to a cluster with identifier $\mathrm{ID}(v)$. Similarly, node u can repeat this process for each node in its surrounding, deducing the entire cluster. Given an additional label for marking nodes in the separating set X, a node can verify that all the TS partition properties hold.

The only real cost of this scheme is describing the radius for each node. One natural approach to reduce the cost is to encode this information across each cluster, however there is no guarantee the clusters are sufficiently large to ensure it can be stored by constant size labels. We instead take a different approach: We observe that sharing some common string S across the graph is much cheaper than assigning each node v some distinct string S_v of its own (See [OPR17, FFH+21, FOS21]). Therefore, instead of describing the radii explicitly, we would like some shared string from which all nodes could deduce their encoded radius, as well as the radii of nodes in their $O(t \log n)$-neighborhood.

As our key observation we show that picking a *random radius* for each node is good, assuming additional alterations to the process. Therefore, assuming all nodes have access to a long shared randomness string, they can deduce the radius of every node. We then condense this string to $O(\log n)$ bits using randomness reduction techniques, and have the prover share a good random string for our graph G across all nodes. We describe the new process, and explain how to identify clusters.

Solution - Ball carving with randomized radius: Assume we have a given random string R. The new process runs in iterations $i = 1, \ldots, n$, where we iterate over the vertices in lexicographic order while maintaining the set of currently living (unclustered) nodes L_i, where we iterate over v_i even

if $v_i \notin L_i$. In step i, we check whether v_i has a living node within distance $2t \log n$. If it does, it samples a random even radius $r_i \in [2t \log n + 2, 8t \log n]$ using R, and considers the candidate cluster $C_i = B_{r_i}(v_i) \cap L_i$ (which does not contain v_i if $v_i \notin L_i$). As before, we take the cluster $C_i = B_{r_i}(v_i) \cap L_i$ if it does not expand, i.e. if

$$|B_{r_i}(v_i) \cap L_i| / |B_{r_i-2}(v_i) \cap L_i| \leq 1 + 1/t.$$

then C_i is carved out of L_i and its 2-boundary with subsequent clusters is added to X. The output is the carved clusters $\mathcal{C}$ together with the marked boundary nodes X, which we show (with high probability) form an $(O(t \log n), 1/t)$-TS partition. Intuitively, we show that in any fixed step i, the cluster C_i is non-expanding with constant probability over the choice of r_i. Moreover, because we iterate over all nodes (even those already clustered), and since every node u is contained in $B_{2t \log n}(v_i)$ for $\Omega(t \log n)$ different nodes v_i, we get that all nodes are clustered with high probability.

Unlike the prior attempt, all a node u needs to know to identify its cluster is (a) the randomness string, (b) for every node v in its vicinity, whether its cluster was taken, and (c) whether v is in X. All this information can be shared with v using $O(1)$ bits, using some randomness reduction technique, and a dedicated PLS.

Refutation of the stronger variant of the Tradeoff Conjecture: In Section 6, we show that having $\Theta(n)$-sized t-neighborhoods is insufficient for obtaining a tradeoff better than $O(\lceil p/t \rceil)$, even for the case of planar graphs, refuting the stronger variant of the Tradeoff Conjecture as discussed above. We do so by defining our predicate P as an equality problem between distant nodes, and reduce it to 2-player non-deterministic equality.

Our construction is very simple. We design a layered graph with $2t + 3$ layers, where odd layers have one node, and even layers have m nodes, with the nodes of each layer connected to all nodes of the neighboring layers (See Figure 2). The leftmost and rightmost nodes, v^1 and v^{2t+3}, receive inputs $I(v^1)$ and $I(v^{2t+3})$ from $\{0,1\}^{m^2}$, and we define the predicate P to assert that $I(v^1) = I(v^{2t+3})$. We design a 1-PLS where even layers relay and split $I(v^1)$ into m parts, each part given to some node in the layer, which requires a label size $O(m)$ bits per node. We show that a t-PLS on P with cost p implies a 2-player non-deterministic equality protocol of cost $O(p \cdot m \cdot t)$. In combination with the fact that m^2-bit equality costs $\Omega(m^2)$ in the 2-player model, we obtain a lower bound of $\Omega(m/t)$ on the cost of any t-PLS for P.

We remark that one can easily transform P into a pure graph property (i.e. without any auxiliary input) by encoding $I(v^1), I(v^{2t+3})$ through two low-diameter subgraphs with $\Theta(m^2)$ edges connected to v^1, v^{2t+3} respectively. However the graph then is not necessarily planar. For planar graphs, a similar graph property with 1-PLS of cost $O(\log m)$, versus t-PLS of cost $\Omega(\log m/t)$ can be easily shown by encoding two low-diameter graphs with $\Theta(m)$ edges next to each of the nodes v^1, v^{2t+3}.

2 Preliminaries

We denote $[k] = \{1, \ldots, k\}$, and $[k_1, k_2] = \{k_1, \ldots, k_2\}$. Given functions f_1, f_2 with disjoint domains, the combined function is defined as $f = f_1 \cup f_2$, where $f(x) = f_1(x)$ if $x \in \mathrm{Dom}(f_1)$, and $f(x) = f_2(x)$ if $x \in \mathrm{Dom}(f_2)$.

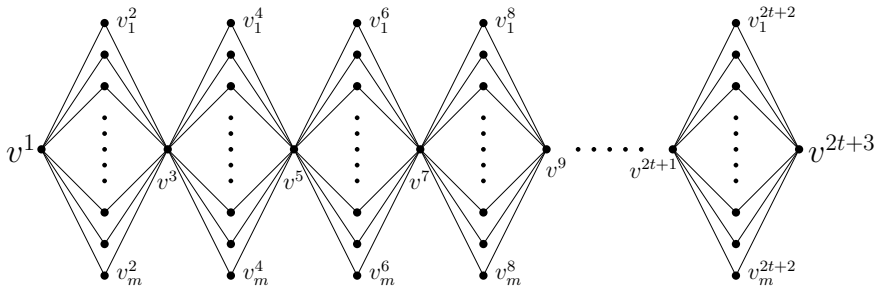


Figure 2: The graph $G_{t,m}$. It has $2t+3$ layers, and in each even layer has m nodes. The leftmost and rightmost nodes of the graph v^1, v^{2t+3} receive as input some strings $I(v^1), I(v^{2t+3})$, and the configuration is in P if $I(v^1) = I(v^{2t+3})$.

Graph notation: For a graph $G = (V, E)$, and a pair of vertices $u, v \in V$, $\mathrm{dist}_G(u, v)$ denotes the shortest path metric in G (that is the minimum number of edges in a u-v path). For $V_1, V_2 \subseteq V$, let $E(V_1, V_2) = \{\{v_i, v_j\} \in E \mid v_i \in V_1, v_j \in V_2\}$ be the edges between V_1, V_2. $G[C] = (C, E(C, C))$ denotes the induced graph on C, i.e. it contains the edge set $\{\{u, v\} \in E \mid u, v \in C\}$. The (strong) diameter of a cluster $C \subseteq V$ is the maximum pairwise distance in the induced graph: $\max_{u,v \in C} \mathrm{dist}_{G[C]}(u, v)$, while the weak diameter is the maximum pairwise distance w.r.t. original distance: $\text{weak-diam}(C) = \max_{u,v \in C} \mathrm{dist}_G(u, v)$. We note that C can have a finite weak diameter while not being connected. A partition $\mathcal{C}$ of V is weakly/strongly Λ-bounded if every cluster has weak/strong diameter at most Λ. Let $N(v) \subseteq V$ denote the neighborhood of v. For an integer $t \geq 1$, let $B_t(v) = \{u \in V \mid \mathrm{dist}_G(u, v) \leq t\} \subseteq V$ be the t-hop neighborhood of v in G, also called the ball of radius t around v. For a set $V' \subseteq V$, let $B_t(V') = \bigcup_{v \in V'} B_t(v)$.

A graph H is said to be a *minor* of G if H can be obtained from G by a series of vertex deletions, edge deletions, and edge contractions.[8] The family of H-minor free graphs consists of all graphs G that do not contain H as a minor (i.e., no series of operations leads from G to H).

String notation: Given a collection of strings $S_1, \ldots, S_\ell$, let $(S_1, \ldots, S_\ell)$ denote a single string obtained by concatenating $S_1, \ldots, S_\ell$ using a fixed delimiter scheme, so that from $(S_1, \ldots, S_\ell)$ one can uniquely recover the number of strings ℓ and each of the strings $S_1, \ldots, S_\ell$. With some abuse of notation, we use $(S_1, \ldots, S_\ell)$ to denote both the tuple of strings and its encoded representation as a single string; the intended meaning will always be clear from context. For an integer p, let $\{0,1\}^{\leq p}$ denote the set of all binary strings of length at most p. For an integer m, let $\mathrm{Pad}(S, m)$ denote an encoding of S as a string whose length is the smallest multiple of m that is larger than $|S|$, such that S can be uniquely recovered from $\mathrm{Pad}(S, m)$. We remark that both the delimiter scheme and the padding scheme can be trivially implemented by implementing a larger alphabet over the binary encoding.

Proof Labeling Schemes: For a graph family $\mathcal{G}$ and set of strings $\mathcal{I}$, a $(\mathcal{G}, \mathcal{I})$-*configuration* is a pair (G, I) where $G \in \mathcal{G}$ is a graph, and $I : V \to \mathcal{I}$ is an assignment of inputs to the nodes of G. A typical example of additional inputs I that can be given to nodes is weights on incident edges, such as in the MST problem [KK07], or shortest path problems [FFH+21].

[8] Given a graph $G = (V, E)$ and an edge $e = \{u, v\} \in E$, by contracting e we obtain a new graph G' where u, v are contracted into a new single vertex w, such that w is adjacent to $(N(u) \cup N(v)) \setminus \{u, v\}$.

Let $(\mathcal{G},\mathcal{I})$ be the family of all $(\mathcal{G},\mathcal{I})$-configurations, for some graph family $\mathcal{G}$ and set of strings $\mathcal{I}$. For a predicate $P : \mathcal{G} \times \mathcal{I} \to \{0,1\}$, we say that $(G,I) \in P$ if $P(G,I) = 1$.

Definition 1. *Let $(\mathcal{G},\mathcal{I})$ be a configuration family. Then for $t \geq 1$, a t-hop proof labeling scheme (t-PLS) for a predicate $P : \mathcal{G} \times \mathcal{I} \to \{0,1\}$ is a pair $\Pi = (\mathrm{Prov}, \mathrm{Ver})$ satisfying*

- Prov*, also called the honest prover, is a mapping taking as input a configuration $(G,I) \in P$, and producing a label assignment $\mathrm{Prov}(G,I) = (\ell(v))_{v\in V}$ for all nodes of G, where $\ell(v) \in \{0,1\}^*$ for each $v \in V$.*
- Ver*, also called the verifier, is a function receiving as input the size of the graph $|V(G)| = n$, and all the information in the t-hop neighborhood of v, including the t-hop topology of v (with the corresponding identifiers), and for each node $u \in B_t(v)$, its additional inputs $I(u)$ and label assignments $\ell(u)$, and outputs a value in $\{\mathrm{accept}, \mathrm{reject}\}$ (i.e.* accepts *or* rejects *respectively).*[9] *Let $\mathrm{Ver}(G,I,\ell,v)$ denote the output of the function at node $v \in V$, in configuration (G,I), and with labels $\ell = (\ell(v))_{v\in V}$. If* Ver *accepts, we also allow it to output additional auxiliary values.*

Additionally, the following holds:

- *Completeness: for every configuration $(G,I) \in P$ and for every node $v \in V$, we have that $\mathrm{Ver}(G,I,\mathrm{Prov}(G,I),v) = \mathrm{accept}$.*
- *Soundness: for every configuration $(G,I) \in (\mathcal{G},\mathcal{I})$ and certificate assignment $\ell = (\ell(v))_{v\in V}$, if $\mathrm{Ver}(G,I,\ell,v) = \mathrm{accept}$ for all $v \in V$, then $(G,I) \in P$.*

We denote the label given to a node v by $\mathrm{Prov}(G,I)$ as $\mathrm{Prov}(G,I,v)$. The *cost* of a t-PLS $(\mathrm{Prov},\mathrm{Ver})$ on a configuration $(G,I) \in P$ is defined as $\mathrm{cost}(\mathrm{Prov},G,I) = \max_{v\in V} |\mathrm{Prov}(G,I,v)|$, i.e. the length of the longest label given to a node by $\mathrm{Prov}(G,I)$. The cost of a t-PLS $(\mathrm{Prov},\mathrm{Ver})$ on the configuration family $(\mathcal{G},\mathcal{I})$ is defined by

$$\mathrm{cost}(\mathrm{Prov},P,n) = \max\{\mathrm{cost}(\mathrm{Prov},G,I) \mid (G,I) \in P, |V(G)| = n\}.$$

Verifier knowledge and additional assumptions: Throughout the paper, we assume that nodes have unique identifiers encoded using $O(\log n)$ bits. We further assume that Ver is given access to the size of the graph, i.e., $|V(G)| = n$, and that the graph is connected. These latter two assumptions are only used in Section B, and we believe they can be removed given some additional technical details, as briefly discussed at the end of that section.

3 Two-Separated Partitions and Cluster-Degeneracy

In this section, we present two key notions: Two-Separated partitions (TS partitions) and ε-cluster-degeneracy, and relate these notions to obtaining tradeoff theorems. We start by introducing TS partitions, which are our key combinatorial objects whose existence implies the tradeoff theorems.

Definition 2 (TS partition)**.** *A (t,ε)-*Two-Separated partition *(TS partition) of a graph $G = (V,E)$ is a pair $(\mathcal{C},X)$ where $X \subseteq V$, and $\mathcal{C}$ is a partition of V, such that:*

[9] Equivalently, Ver can be seen as a t-round deterministic distributed algorithm with unlimited bandwidth that collects all information in the t-neighborhood of v, and outputs a value in $\{\mathrm{accept}, \mathrm{reject}\}$, and additional output.

- Two separation. $\forall C, C' \in \mathcal{C}$, *every path* $P = (v_1, \ldots, v_\ell)$ *from a vertex in* $C \setminus X$ *to a vertex in* $C' \setminus X$ *has two consecutive vertices in* X. *That is, for some* $i \in [\ell - 1]$, $v_i, v_{i+1} \in X$.
- Bounded diameter. $\forall C \in \mathcal{C}$, weak-diam$(C) \leq t$. *We say that a TS partition is* connected *if every* $C \in \mathcal{C}$ *is connected.*
- *The* cost ratio *of* $(\mathcal{C}, X)$, *defined as* $R(\mathcal{C}, X) = \max_i |C_i \cap X|/|C_i|$, *holds* $R(\mathcal{C}, X) \leq \varepsilon$.

Similarly, define for a TS partition $(\mathcal{C}, X)$ *the cost ratio of a cluster* $C \in \mathcal{C}$ *as* $|C \cap X|/|C|$.

Next, we introduce the notion of a decomposition into clusters with ε-cluster-degeneracy, and prove that when combined with low-diameter constraints, the existence of such a decomposition guarantees the existence of a (t, ε)-TS partition. This notion provides a convenient approach for constructing TS partitions in graphs, which we use in both Section 4 and Section 5.

Given a ordered partition $\mathcal{C} = \{C_1, \ldots, C_k\}$, a node $v \in C_i$ is called a *close-to-boundary* node if there is a node $u \in \cup_{j>i} C_j$ in a cluster with a higher index than i, at distance at most 2. We say that $\mathcal{C}$ has ε-cluster-degeneracy if in every cluster C_i, at most ε-fraction of its nodes are close-to-boundary nodes.

Definition 3. *A partition of* V *into clusters* $\mathcal{C} = \{C_1, \ldots, C_k\}$ *has* ε*-cluster-degeneracy if*

$$\forall i, \quad \left|\left\{v \in C_i \mid \mathrm{dist}_G\left(v, V \setminus \cup_{j=1}^{i} C_j\right) \leq 2\right\}\right| \leq \varepsilon \cdot |C_i|.$$

Lemma 3.1. *Assume a graph* $G = (V, E)$ *has a partition into clusters* $\mathcal{C} = \{C_1, \ldots, C_k\}$ *that has* ε*-cluster-degeneracy and that is weakly* t*-bounded. Then* G *has a* (t, ε)*-TS partition. Moreover, if all clusters in* $\mathcal{C}$ *are connected, then* G *has a connected* (t, ε)*-TS partition.*

Proof. Let $V_{>i} = V \setminus \cup_{j=1}^{i} C_j$, and $V_{>k} = \emptyset$. Setting $X = \cup_{i=1}^{k} (C_i \cap B_2(V_{>i}))$ we argue that $(\mathcal{C}, X)$ is a (t, ε)-TS partition, i.e. it fulfills the following three properties:

- *Diameter and connectivity:* By assumption weak-diam$(C) \leq t$ for all $C \in \mathcal{C}$. Additionally, the partition is connected if and only if each $C \in \mathcal{C}$ is connected.
- *Cost ratio:* We note that $C_i \cap X = C_i \cap B_2(V_{>i})$, since $C_i \cap B_2(V_{>i}) \subseteq C_i$, and the clusters are disjoint. Therefore, by assumption we have $|C_i \cap X|/|C_i| = |C_i \cap B_2(V_{>i})|/|C_i| \leq \varepsilon$.
- *Two Separation:* Assume for the sake of contradiction that the two separation property fails. Let $P_{u,v}$ be the shortest path between two vertices $u, v \notin X$ that belong to different clusters $\mathcal{C}$, such that $P_{u,v}$ does not have two consecutive vertices from X.

  Denote $P_{u,v} = (w_0 = u, w_1, \ldots, w_{\ell-1}, w_\ell = v)$. Let $u \in C_i, v \in C_j$ where we assume w.l.o.g. that $i < j$. Since $u \notin X$, we have $\mathrm{dist}(u, V_{>i}) > 2$, and specifically we have $\ell \geq 3$ and $w_1, w_2 \notin C_j$.

  If $w_1 \notin X$, then $P_{w_1,v} = (w_1, \ldots, w_\ell = v)$ contradicts the minimality of $P_{u,v}$, thus $w_1 \in X$. Similarly, we deduce that $w_2 \in X$. Since $w_1, w_2 \in X$ are consecutive on $P_{u,v}$, we obtain a contradiction, and the claim follows.

□

In the remainder of the section, we show that the existence of (t,ε)-TS partitions implies a tradeoff between $O(t)$-PLS and 1-PLS. To set this up, we first define the verification task of outputting a (t,ε)-TS partition, in which we both verify the existence of a (t,ε)-TS partition, and have each node locally output nearby clusters and their nodes in X.

Definition 4. *We say that a t-PLS* $\Pi = (\mathrm{Prov}, \mathrm{Ver})$ *certifies and outputs a* (t,ε)*-TS partition if* Π *is a t-PLS for the existence of a* (t,ε)*-TS partition in the graph according to Definition 1, and require an additional guarantee: assuming no node rejects for some labeling* ℓ*,* Ver *outputs for each node* v *a set of clusters* $\mathrm{Comp}(v)$ *and a set* $X_C(v) \subseteq V$ *for each* $C \in \mathrm{Comp}(v)$*, such that there exists a* (t,ε)*-TS partition* $(\mathcal{C}, X)$ *of* G*, satisfying the following properties:*

(1) Local cluster knowledge. *For any node* $v \in V$*, let* $C_v \in \mathcal{C}$ *denote the unique cluster containing* v*. Then we have for any* $v \in V$ *that* $\mathrm{Comp}(v) = \{C_u \mid u \in B_2(C_v)\}$*. In other words, each node* v *knows* C_v *and any cluster that is of distance at most two from* C_v*.*

(2) Local boundary knowledge. *For any* $v \in V$*, we have* $X_C(v) = X \cap C$ *for any* $C \in \mathrm{Comp}(v)$*.*

In this case, we refer to $(\mathcal{C}, X)$ *as the TS partition outputted by the labeling* ℓ*.*

Finally, we formalize the relationship between the existence of TS partitions and a tradeoff theorem. Specifically, we show that a (t,ε)-partition that can be outputted with cost c implies that any 1-PLS of cost p can be transformed into an $O(t)$-PLS with cost $O(\lceil \varepsilon \cdot p \rceil + c)$.

Lemma 3.2 (TS partition implies tradeoff)**.** *Let* $(\mathcal{G}, \mathcal{I})$ *be a configuration family. Suppose that for some functions* $t = t(n) \geq 1$ *and* $\varepsilon = \varepsilon(n) \in [0,1)$*, every graph* $G \in \mathcal{G}$ *with* $|V(G)| = n$ *admits a* (t,ε)*-TS partition, and that such a partition can be certified and outputted by an* $O(t)$*-PLS with cost* $c = c(n)$*. Then, for any predicate* P *on* $(\mathcal{G}, \mathcal{I})$ *and any* 1*-PLS for* P *with cost* p*, there exists an* $O(t)$*-PLS for* P *with cost* $O(\lceil \varepsilon \cdot p \rceil + c)$*.*

Before presenting the proof, we analyze basic properties of TS partitions, and simple lemmas related to certification of P. Then, we discuss efficient encoding and retrieval of information from clusters, which is a crucial part of the prover and verifier algorithms.

Basic properties and labeling of (t,ε)-TS partitions We start with a basic claim, that for every node v, there is at most a single cluster C such that $B_1(v)$ (v and its neighborhood) contains non-boundary vertices from C: $C \setminus X$.

Lemma 3.3. *Let* $(\mathcal{C}, X)$ *be a* (t,ε)*-TS partition. Then for every* $v \in V$*, there exists a cluster* $C \in \mathcal{C}$ *such that* $B_1(v) \subseteq C \cup X$*. In particular,* C *is unique unless* $B_1(v) \subseteq X$*.*

Proof. Suppose for the sake of contradiction that there is a vertex v such that $B_1(v)$ is not contained in $C \cup X$ for any $C \in \mathcal{C}$ (in particular $B_1(v) \not\subseteq X$). It follows that there are two neighbors u_1, u_2 of v, such that $u_1 \in C_1 \setminus X$ and $u_2 \in C_2 \setminus X$ for some clusters $C_1 \neq C_2 \in \mathcal{C}$. But then (u_1, v, u_2) is a 2-path connecting $C_1 \setminus X$ and $C_2 \setminus X$, contradiction to the two separation property. Uniqueness follows from the fact that clusters are disjoint, hence if $B_1(v) \subseteq C_1 \cup X$ and $B_1(v) \subseteq C_2 \cup X$ for some distinct clusters $C_1 \neq C_2$, then it follows that $B_1(v) \subseteq X$. □

Define $H_C = \{v \in V \mid B_1(v) \cap (C \setminus X) \neq \emptyset\}$. We have the following two properties of H_C.

Lemma 3.4. *The following two properties hold.*

(1) $\bigcup_{C \in \mathcal{C}} H_C$ *is equal to the set* $\{u \mid B_1(u) \not\subseteq X\}$.

(2) $H_C \cap H_{C'} = \emptyset$ *for any two distinct clusters* $C, C' \in \mathcal{C}$.

Proof. We prove the two properties above.

(1) This follows trivially from the definition of $H_C = \{v \in V \mid B_1(v) \cap (C \setminus X) \neq \emptyset\}$.

(2) Let $u \in H_C$. Then by Lemma 3.3 we have $B_1(u) \cap (C' \setminus X) = \emptyset$, or in other words $u \notin H_{C'}$.

□

Let $\Pi_1 = (\text{Prov}_1, \text{Ver}_1)$ be a 1-PLS for a property P. We introduce two notions for partial labelings related to Π_1 which help us certify that $(G, I) \in P$ using an $O(t)$-PLS. Namely, we define some notion of a good labeling of nodes of X (according to Π_1), and for every C, given a good labeling of nodes of X, we define the notion of a good extension of the good labeling into $C \setminus X$. We then show that if there exists a good labeling of X, and a good extension of the labeling into $C \setminus X$ for every $C \in \mathcal{C}$, then $(G, I) \in P$.

Definition 5. *Let* $\Pi_1 = (\text{Prov}_1, \text{Ver}_1)$ *be a 1-PLS for a property* P. *Let* $(\mathcal{C}, X)$ *be a* (t, ε)*-TS partition. We define the following.*

- *We say that a labeling* $\ell_X : X \to \{0,1\}^{\leq p}$ *is a good* X*-labeling if for all nodes* $v \in X$ *such that* $B_1(v) \subseteq X$, *we have* $\text{Ver}_1(G, I, \ell_X, v) = \text{accept}$.
- *Let* ℓ_X *be a good* X*-labeling. For a cluster* $C \in \mathcal{C}$, *a function* $\ell_{C \cup X} : (C \cup X) \to \{0,1\}^{\leq p}$ *is called a good extension of* ℓ_X *if* $\ell_{C \cup X}(u) = \ell_X(u)$ *for every* $u \in X$, *and for every* $v \in H_C$ *we have* $\text{Ver}_1(G, I, \ell_{C \cup X}, v) = \text{accept}$.

Lemma 3.5. *Let* $\Pi_1 = (\text{Prov}_1, \text{Ver}_1)$ *be a 1-PLS for a property* P. *Let* $(\mathcal{C}, X)$ *be a* (t, ε)*-TS partition, and let* $\ell_X : X \to \{0,1\}^{\leq p}$ *be a good* X*-labeling. If for every cluster* $C \in \mathcal{C}$ *there exists a good extension* $\ell_{X \cup C}$, *then* $(G, I) \in P$.

Proof. Let ℓ_X be a good X-labeling, and let $\ell_{C_1 \cup X}, \ldots, \ell_{C_k \cup X}$ be good extensions of ℓ_X for clusters $C_1, \ldots, C_k$, which exist by the lemma's assumption. We define a labeling $\ell_V : V \to \{0,1\}^{\leq p}$ as follows.

$$\ell_V(u) = \begin{cases} \ell_X(u), & u \in X, \\ \ell_{C \cup X}(u), & u \in C \setminus X. \end{cases}$$

We argue that $\text{Ver}_1(G, I, \ell_V, u) = \text{accept}$ for all $u \in V$, which implies that $(G, I) \in P$ by the soundness of $(\text{Prov}_1, \text{Ver}_1)$. We split into two cases - when $B_1(u) \subseteq X$ and otherwise. In the first case, we have by definition of ℓ_X being good that $\text{Ver}_1(G, I, \ell_V, u) = \text{Ver}_1(G, I, \ell_X, u) = \text{accept}$. In the other case, we have by Lemma 3.4 that $u \in H_C$ for some unique $C \in \mathcal{C}$. Therefore, by definition of $\ell_{C \cup X}$ being a good extension of ℓ_X, we get $\text{Ver}_1(G, I, \ell_V, u) = \text{Ver}_1(G, I, \ell_{X \cup C}, u) = \text{accept}$. □

Finally, we discuss how to encode a string across a cluster using node labels, and how to decode it again in order to retrieve the stored information.

Encoding and decoding strings on clusters. As motivated in the introduction, we would like to be able to encode a good ℓ_X on the labels of the network, where $\ell_X(v)$ for $v \in C \cap X$ is encoded using labels of cluster C's vertices. We first discuss encoding and decoding strings across a cluster, which uses the internal identifier ordering to describe the part numbers of string segments spread across the cluster.

Given a cluster $C \subseteq V$ and a string S, we define the lexicographic encoding Lex of S on C as follows. Let $k = |C|$, and let $u_1, \ldots, u_k$ be the nodes of C, ordered lexicographically by their node identifiers. Let $S' = \text{Pad}(S, k)$ be a padding of S so that $|S'|$ is divisible by k, and let $S'_1, \ldots, S'_k$ denote the strings obtained by partitioning S' into k substrings, each of the same length. For node u_i, we define $\text{Lex}(C, S, u_i) = S'_i$.[10]

Relatedly, we also define a decoding function DecodeLex. Given a cluster C and a labeling ℓ on its vertices such that $\ell(u_i) = \text{Lex}(C, S, u_i)$ for some string S, the function $\text{DecodeLex}(C, \ell)$ reconstructs S by concatenating the labels in lexicographic order and then removing the padding.

We are now ready to prove Lemma 3.2.

Proof of Lemma 3.2. Given $(\text{Prov}_{\text{TS}}, \text{Ver}_{\text{TS}})$, an $O(t)$-PLS for computing a (t, ε)-TS partition, and $(\text{Prov}_1, \text{Ver}_1)$ a 1-PLS for certifying P, we construct $(\text{Prov}_t, \text{Ver}_t)$ as follows. We assume w.l.o.g. that the labels given by Prov_t and received by Ver_t for each node are of the form $(\text{TSLabel}, \text{ProofPart})$ where $\text{TSLabel}, \text{ProofPart}$ are some strings. If a label breaks this format, Ver_t rejects in the corresponding node.

Description of Prov_t: Let $(\mathcal{C}, X)$ be a (t, ε)-TS partition of the graph that is outputted by Prov_{TS} on the graph G. For a node v, denote C_v as the unique cluster containing v. Let $u_1, \ldots, u_k$ be the nodes of $C_v \cap X$, ordered lexicographically w.r.t. their node identifier, and let

$$\text{XLabel}(C_v, X) = (\text{Prov}_1(G, I, u_1), \ldots, \text{Prov}_1(G, I, u_k)).$$

The mapping Prov_t gives each node v as its label the tuple

$$\text{Prov}_t(G, I, v) = (\text{Prov}_{\text{TS}}(G, I, v) \ , \ \text{Lex}(C_v, \text{XLabel}(C_v, X), v)) \, .$$

By the lemma's assumption, $\text{Prov}_{\text{TS}}(G, I, v)$ is of size at most c. For any $C \in \mathcal{C}$ we have $|\text{XLabel}(C, X)| \leq p \cdot |C \cap X|$. Therefore,

$$|\,\text{Lex}(C_v, \text{XLabel}(C_v, X), v)| = O(\lceil p \cdot |C_v \cap X|/|C_v| \rceil) = O(\lceil p \cdot \varepsilon \cdot |C_v|/|C_v| \rceil) = O(\lceil \varepsilon \cdot p \rceil).$$

Hence the total cost of Prov_t is $O(\lceil \varepsilon \cdot p \rceil + c)$ as required.

Description of Ver_t: Recall we assume that the proof labels of every node v are given in a format encoding two arbitrarily large strings $(\text{TSLabel}(v), \text{ProofPart}(v))$ (otherwise we have that node reject). Let $\text{TSLabel} : V \to \{0,1\}^*, \text{ProofPart} : V \to \{0,1\}^*$ be the corresponding global functions/labelings. We describe Ver_t as a distributed algorithm, where we say that a node accepts or rejects corresponding to whether Ver_t accepts or rejects on this node.

In the first step of the verifier we have every node v run Ver_{TS} on the labeling TSLabel to certify and output some (t, ε)-TS partition. By the soundness of Π_{TS}, either a node rejects, or there exists some (t, ε)-TS partition $(\mathcal{C}, X)$ satisfying the properties of Definition 4. Assuming no node rejects,

[10] We refer back to the string notations in Section 2 for the introduction of Pad and the fixed delimiter scheme.

we fix this partition $(\mathcal{C}, X)$. Let C_v denote the unique cluster containing v, and denote for each cluster $C \in \mathcal{C}$ by leader(C) as the node with the maximum node identifier in $C \setminus X$.

From the labeling ProofPart we globally define a labeling function $\ell_X : X \to \{0,1\}^*$ for all nodes in X, in the following manner. Let $C \in \mathcal{C}$ be a cluster, and let $C \cap X = \{u_1, \ldots, u_k\}$ be the nodes of $C \cap X$ ordered lexicographically by node id. Then we define,

$$(\ell_X(u_1), \ldots, \ell_X(u_k)) = \text{DecodeLex}(C, \text{ProofPart}).$$

Node $v = \text{leader}(C)$ asserts that $\text{DecodeLex}(C, \text{ProofPart})$ does indeed encode $k = |C \cap X|$ strings, rejecting otherwise.

We note that to compute $\ell_X(v)$ of any node $v \in C \cap X$ it is sufficient to know the topology of C with its identifiers, and its labels of ProofPart. Therefore, any node v can locally reconstruct for each of $C \in \text{Comp}(v)$ the labels $\ell_X(u)$ of all nodes $u \in C \cap X$, and specifically for $u \in B_2(C_v) \cap X$.

Recall the definition of $H_C = \{v \mid B_1(v) \cap (C \setminus X) \neq \emptyset\}$. Every node v performs the following.

(1) If $B_1(v) \subseteq X$ (including v), node v asserts that $\text{Ver}_1(G, I, \ell_X, v) = \text{accept}$.

(2) If $v = \text{leader}(C)$ for some cluster $C \in \mathcal{C}$, node v asserts that there exists a labeling $\ell_{C \setminus X} : C \setminus X \to \{0,1\}^{\leq p}$ such that the extension $\ell_{C \cup X} = \ell_X \cup \ell_{C \setminus X}$ is good, or in other words that $\forall_{u \in H_C} \text{Ver}_1(G, I, \ell_{C \cup X}, u) = \text{accept}$. We note that leader$(C)$ can perform this since in order to check that the extension is good, leader(C) only needs to know $\ell_X(u)$ for $u \in B_2(C) \cap X$.

(3) If all assertions passed, node v accepts.

Each node checks the assertions above, and accepts if and only if no assertion fails. This concludes the description of Ver_t. We summarize all assertions made by the verifier in the following list, from the viewpoint of a node v (skipping some in-between steps).

(1) Node v runs Ver_{TS} on the labeling TSLabel, and rejects if it rejects. Otherwise, it obtains its relevant part in $(\mathcal{C}, X)$, i.e. $\text{Comp}(v) \subseteq \mathcal{C}$ and for each $C \in \text{Comp}(v)$ a set $X_C(v)$.

(2) If $v = \text{leader}(C)$ for some $C \in \text{Comp}(v)$, it asserts whether $\text{DecodeLex}(C, \text{ProofPart})$ encodes $|C \cap X|$ many strings (which correspond to $(\ell_X(u))_{u \in C \cap X}$).

(3) If $B_1(v) \subseteq X$ (including v), node v asserts that $\text{Ver}_1(G, I, \ell_X, v) = \text{accept}$.

(4) If $v = \text{leader}(C)$ for some $C \in \text{Comp}(v)$, then it asserts that there exists a labeling $\ell_{C \setminus X}$ of $C \setminus X$ such that the extension $\ell_{C \cup X} = \ell_X \cup \ell_{C \setminus X}$ is good, i.e. that $\forall_{u \in H_C} \text{Ver}_1(G, I, \ell_{C \cup X}, u) = \text{accept}$.

Correctness: To show correctness, we assume that a (t, ε)-TS partition exists, and show that $\text{Ver}_t(G, I, \text{Prov}_t(G, I), v) = \text{accept}$, i.e. that all nodes accept given the honest prover labeling.

We have that $\text{TSLabel}(v) = \text{Prov}_{\text{TS}}(G, I, v)$, therefore by correctness of $(\text{Prov}_{\text{TS}}, \text{Ver}_{\text{TS}})$ no node rejects, and we have that each node obtains a set $\text{Comp}(v) \subseteq \mathcal{C}$ and X_C for each $\text{Comp}(v)$ satisfying the guarantees of Definition 4, passing Assertion (1).

Recall the definition of $\text{XLabel}(C, X) = (\text{Prov}_1(G, I, u))_{u \in C \cap X}$. For each node v, we have that $\text{ProofPart}(v) = \text{Lex}(C, \text{XLabel}(C_v, X), v)$, therefore for each $C \in \text{Comp}(v)$ we have that

$$\text{DecodeLex}(C, \text{ProofPart}) = (\text{Prov}_1(G, I, u))_{u \in C \cap X}.$$

In particular, we do not reject in Assertion (2), and for every node $v \in V$, we have that it can compute $\ell_X(u)$ for all $u \in C \cap X$.

Finally, we show that a node v does not reject in Assertion (3),(4). For Assertion (3), we note that all of $u \in B_1(v)$ have $\ell_X(u) = \text{Prov}_1(G, I, u)$, and therefore v does not reject, by correctness of Prov_1. For Assertion (4), we note that by correctness of Prov_1, we have that ℓ_X is a good labeling, and hence the labeling $\ell_{C \cup X}(u) = \text{Prov}_1(G, I, u)$ is a good extension to ℓ_X, and $v = \text{leader}(C)$ accepts.

Soundness: Assume that all nodes accept. By Assertion (1),(2) we have some TS partition $(\mathcal{C}, X)$ which the nodes collectively hold, and that each cluster $C \in \mathcal{C}$ encodes $|C \cap X|$ many strings, corresponding to some labeling $(\ell_X(u))_{u \in C \cap X}$. Therefore, the labeling ℓ_X is well-defined. By Assertion (3), we have that ℓ_X is a good X-labeling according to Definition 4, and moreover by Assertion (4), we have for each $C \in \mathcal{C}$ a good extension $\ell_{C \cup X}$. Therefore, by Lemma 3.5, we have that $(G, I) \in P$, and the claim follows.

□

4 Tradeoff Theorem for Minor-Free Graphs

In this section, our goal is to prove that the Tradeoff Conjecture holds for K_r-minor free graphs $\mathcal{G}$ for any constant $r \geq 1$, up to a single additive $O(\log n)$ factor. This trivially implies the conjecture for any H-minor free graph family, since it is K_r-minor free as well, for $r = |V(H)|$.

Theorem 1.4. *Let $r \geq 1$ be a constant, and let $(\mathcal{G}, \mathcal{I})$ be a configuration family where $\mathcal{G}$ is the family of all K_r-minor free graphs, and let P be a predicate on $(\mathcal{G}, \mathcal{I})$. If there exists a 1-PLS for P in $(\mathcal{G}, \mathcal{I})$ with cost p, then for any $t \geq 1$, there exists a t-PLS for P in $(\mathcal{G}, \mathcal{I})$ with cost $O(\lceil p/t \rceil + \log n)$.*

On a high level, we show Theorem 1.4 using the existence of a good padded decomposition (introduced next) in $\mathcal{G}$, and relating its existence to the existence of a good TS partition. Then we use the fact that $O(\log n)$ is always sufficient for an $O(t)$-PLS to certify and output a TS partition.

Definition 6 (Padded Decomposition)**.** *Let $G = (V, E)$ be a graph. A distribution $\mathcal{D}$ over partitions of V is a (β, δ, Λ)-padded decomposition if every $\mathcal{P} \in \text{supp}(\mathcal{D})$ is weakly Λ-bounded, and for every $0 \leq \gamma \leq \delta$ and $z \in V$, the ball $B_{\gamma\Lambda}(z)$ satisfies $\Pr[B_{\gamma\Lambda}(z) \subseteq C_z] \geq e^{-\beta\gamma}$ (C_z denotes the unique cluster in $\mathcal{P}$ containing z). We say that G admits a (β, δ)-padded decomposition scheme if for every $\Lambda > 0$, there is a (β, δ, Λ)-padded decomposition for G.*

We use a result by Conroy and Filtser [CF25], that shows that K_r-minor free graphs have a good padded decomposition.

Theorem 4.1 ([CF25])**.** *For any $r \geq 1$, every K_r-minor free graph admits an $(O(\log r), O(1))$-padded decomposition scheme.*

Next, we show that having a good padded decomposition implies a good TS partition. First, we show that given an "active" set of nodes L, we can construct a cluster $C \subseteq L$ such that all vertices, other than a $O(\beta/t)$ fraction, are at least of distance 2 from all other nodes in L. We then use this iteratively to get a partition with $O(\beta/t)$-cluster-degeneracy, and conclude the existence of a $(t, O(\beta/t))$-TS partition.

Lemma 4.2. *Let $G = (V, E)$ be a graph that admits a $(\beta, O(1))$-padded decomposition scheme. Let $L \subseteq V$ be a set of nodes. Then for any $t = \Omega(1)$, there is a cluster $C \subseteq L$ that has weak diameter $\leq t$, and such that $|C \cap B_2(L \setminus C)| \leq \frac{2\beta}{t} \cdot |C|$.*

Proof. We sample a partition $\mathcal{P} = \{C_1, \ldots, C_k\}$ from a $(\beta, O(1), t)$-padded decomposition. Denote C_v the unique cluster containing v. Let $\Gamma_2(L) = \{v \in L \mid B_2(v) \not\subseteq C_v\}$. We note that $\Gamma_2(L)$ is precisely the set of second-layer boundary vertices in L across all clusters of $\mathcal{P}$. For every vertex v, it holds that

$$\Pr[v \in \Gamma_2(L)] \leq \Pr[B_2(v) \not\subseteq C_v] \leq 1 - e^{-\frac{2\beta}{t}} \leq \frac{2\beta}{t} .$$

Thus by linearity of expectation, $\mathbb{E}[|\Gamma_2(L)|] \leq \frac{2\beta}{t} \cdot |L|$. In particular, there is some partition $\mathcal{P}$ where $|\Gamma_2(L)| \leq \frac{2\beta}{t} \cdot |L|$. Let $\mathcal{P}$ be such a partition, and let $\mathcal{C}$ be the resulting clusters, and let $\mathcal{C}' = \{C \cap L \mid C \in \mathcal{C}\}$. Let $C'_1, \ldots, C'_k$ denote the non-empty clusters of $\mathcal{C}'$. Then the following holds,

$$\min_{i \in [k]} \frac{|C'_i \cap B_2(L \setminus C'_i)|}{|C'_i|} \leq \frac{\sum_{i=1}^{k} |C'_i \cap B_2(L \setminus C'_i)|}{\sum_{i=1}^{k} |C'_i|} \leq \frac{|\Gamma_2(L)|}{|L|} \leq \frac{2\beta}{t} .$$

Where the first inequality follows from the mediant inequality.[11] Thus taking $C = C'_i$ as the cluster minimizing the expression on the left-hand side, we found a cluster C as desired. □

Lemma 4.3. *Let $G = (V, E)$ be a graph that admits a $(\beta, O(1))$-padded decomposition scheme. Then G has a $(t, 2\beta/t)$-TS partition.*

Proof. Consider the following iterative procedure, where we maintain an active set L, and construct in each step a cluster $C \subseteq L$ using Lemma 4.2, and remove C from L until the set is empty.

Initially, set $L_1 = V$. In step $i \geq 1$, assume we have a set $L_i \subseteq V$, satisfying $|L_i| > 0$. By Lemma 4.2, there exists a cluster C_i of weak diameter $\leq t$, such that $|C_i \cap B_2(L_i \setminus C_i)| \leq 2\beta|C_i|/t$. Define $L_{i+1} = L_i \setminus C_i$. We repeat the process until L_{i+1} is empty.

By a simple induction, we have $L_i = V \setminus \cup_{j=1}^{i-1} C_j$ for all $i \in [k]$. Therefore, the set of obtained clusters $\mathcal{C} = \{C_1, \ldots, C_k\}$ are a partition of V, having each weak diameter $\leq t$, and satisfy

$$\forall_i \frac{|C_i \cap B_2(V \setminus \cup_{j=1}^{i} C_j)|}{|C_i|} = \frac{|C_i \cap B_2(L_i \setminus C_i)|}{|C_i|} \leq 2\beta/t.$$

Therefore, $\mathcal{C}$ has $(2\beta/t)$-cluster-degeneracy, and by Lemma 3.1, G has a $(t, 2\beta/t)$-TS partition. □

By combining Theorem 1.4 and Lemma 4.3, we get the following as corollary.

Corollary 4.4. *For any $r \geq 1$, the family of K_r-minor free graphs has a $(t, \Theta(\log r/t))$-TS partition.*

Next, we show that a (t, ε)-TS partition can always be certified and outputted using an $O(t)$-PLS with cost $O(\log n)$. We remark that this lemma is fairly standard, and similar in spirit to prior works (e.g. [FOS21]).

Lemma 4.5. *In any graph family $\mathcal{G}$ and any $t(n) \geq 1, \varepsilon(n) \in [0, 1)$ there exists a $O(t)$-PLS that certifies and outputs a $(t(n), \varepsilon(n))$-TS partition with cost $O(\log n)$.*

[11] Recall that the mediant inequality states that if $a_1, b_1, \ldots, a_\ell, b_\ell > 0$ are real numbers, then $\min_{i \in [\ell]} \frac{a_i}{b_i} \leq \frac{\sum_{i=1}^{\ell} a_i}{\sum_{i=1}^{\ell} b_i}$.

The proof of Lemma 4.5 is deferred to Section A. We are now ready to prove the main theorem.

Proof of Theorem 1.4. Recall that by Lemma 3.2, it suffices to show that $\mathcal{G}$ has a $(t, \Theta(1/t))$-TS partition, and that it can be certified and outputted by an $O(t)$-PLS with cost $O(\log n)$. The theorem follows immediately by combining Corollary 4.4 and Lemma 4.5. □

As an immediate corollary, we can obtain a weaker version of Theorem 1.2 for general graphs, which costs an additional logarithmic additive factor, using the following theorem of Bartal.

Theorem 4.6 ([Bar96])**.** *Every n-vertex graph admits an $(O(\log n), O(1))$-padded decomposition scheme.*

Again, by Lemma 3.2 combined with Lemma 4.3, Lemma 4.5 and Theorem 4.6, we obtain the following.

Corollary 4.7. *Let $(\mathcal{G}, \mathcal{I})$ be a configuration family, and P a predicate on $(\mathcal{G}, \mathcal{I})$. For $G \in \mathcal{G}$ with $|V(G)| = n$, if there exists a 1-PLS for P with cost p, then for any $t \geq 1$ there exists an $O(t \log n)$-PLS for P with cost $O(\lceil p/t \rceil + \log n)$.*

5 Tradeoff Theorem for General Graphs

Our goal in this section is to show a near resolution of the Tradeoff Conjecture in general graphs, up to a single multiplicative logarithmic factor, and with only constant additive factors. In particular, the cost of this scheme converges to a constant number of bits as t grows.

Theorem 1.2. *Let $(\mathcal{G}, \mathcal{I})$ be a configuration family, and P a predicate on $(\mathcal{G}, \mathcal{I})$. For $G \in \mathcal{G}$ with $|V(G)| = n$, if there exists a 1-PLS for P with cost p, then for any $t \geq 1$ there exists an $O(t \log n)$-PLS for P with cost $O(\lceil p/t \rceil)$.*

5.1 Helpful Sequential Algorithm for Finding a $(O(t \log n), 1/t)$-TS Partition

Before proving Theorem 1.2, we describe and analyze a sequential randomized algorithm $\mathcal{A}$ on a graph G, which obtains a $(O(t \log n), 1/t)$-TS partition with high probability, and is later used as part of the labeling scheme. For ease of notation, we treat $\log n$ as an integer throughout the section, and note that this assumption can be easily removed using appropriate rounding functions.

Description of algorithm $\mathcal{A}$: Let $v_1, \ldots, v_n$ be an ordering of V according to lexicographic order. Initially, we initialize our set of clusters as $\mathcal{C} = \emptyset$, and the set of active nodes as $L_1 = V$. We iterate over the nodes according to the ordering. In step $i \geq 1$, we check whether $L_i \cap B_{2t \log n}(v_i) = \emptyset$, and if so, we continue to step $i + 1$. Otherwise, we choose a random even integer $r_i \in \{2t \log n + 2, 2t \log n + 4, \ldots, 8t \log n\}$, and define $C_i = B_{r_i}(v_i) \cap L_i$. We note that C_i does not necessarily contain v_i, and also might be disconnected. We add C_i to $\mathcal{C}$ if

$$\frac{|B_{r_i}(v_i) \cap L_i|}{|B_{r_i - 2}(v_i) \cap L_i|} \leq 1 + \frac{1}{t} \,. \tag{1}$$

If C_i was added to $\mathcal{C}$, we set $L_{i+1} = L_i \setminus C_i$, set $X_i = C_i \cap B_2(L_i \setminus C_i)$. Otherwise, we set $X_i = \emptyset$ and $L_{i+1} = L_i$. Following this, we continue to step $i + 1$. We conclude at the end of step n, i.e. after the final vertex v_n is processed. Let $X = \bigcup_{i=1}^{n} X_i$. We output $(\mathcal{C}, X)$ as our TS partition.

Analysis of algorithm $\mathcal{A}$: In the analysis, our goal is to show that with some high probability the resulting $(\mathcal{C}, X)$ is an $(O(t\log n), 1/t)$-TS partition of G. Specifically, we need to show that $\mathcal{C}$ is a partition of V (i.e., every node is assigned to exactly one cluster in $\mathcal{C}$), that the clusters are weakly t-bounded, have cost ratio at most $1/t$, and are two separated. We first show that $L_{n+1} = \emptyset$, which implies that $\mathcal{C}$ is a partition. Following this, we show that conditioned that $\mathcal{C}$ is a partition, then $(\mathcal{C}, X)$ is a $(O(t\log n), 1/t)$-TS partition.

We next show that in each step i, if we have some node in L_i in the vicinity of v_i (i.e. $L_i \cap B_{2t\log n}(v_i) \neq \emptyset$), then with some constant probability we create a new cluster in $\mathcal{C}$ that contains these nodes. We later argue that since each node has $\Theta(t\log n)$ nodes in its $\Theta(t\log n)$-neighborhood (unless the graph is smaller than $\Theta(t\log n)$), then with high probability each node is added to a cluster in some step, and the correctness of the algorithm follows.

Lemma 5.1. *If at the start of step i we have $L_i \cap B_{2t\log n}(v_i) \neq \emptyset$, then Eq. (1) holds with probability at least $1/2$ over the choice of $r_i \in [2t\log n + 2, 8t\log n]$.*

Proof. Fix a step i, and denote

$$\mathrm{Bad}_i = \{r \in \{2t\log n + 2, 2t\log n + 4, \ldots, 8t\log n\} \mid r \text{ does not satisfy } Eq.\ (1) \text{ in step } i\}.$$

Assume by contradiction that $|\,\mathrm{Bad}_i\,| \geq t\log n + 1$, and let $\{r'_1, \ldots, r'_{t\log n+1}\} \subseteq \mathrm{Bad}_i$, ordered in ascending order. Therefore, for all $j \in [2, t\log n + 1]$ we have that

$$|B_{r'_j}(v_i) \cap L_i| \geq (1 + \frac{1}{t})|B_{r'_j-2}(v_i) \cap L_i| \geq (1 + \frac{1}{t})|B_{r'_{j-1}}(v_i) \cap L_i|,$$

where the first inequality holds by definition of Bad_i, and the second inequality holds due to the fact that we considered only even indices, hence $r'_{j-1} \leq r'_j - 2$, and the fact that a ball can only increase in size as the radius grows. By induction on j, we get that

$$|B_{r'_{t\log n+1}}(v_i) \cap L_i| \geq (1 + \frac{1}{t})^{t\log n}|B_{r'_1}(v_i) \cap L_i| \geq (1 + \frac{1}{t})^{t\log n}|B_{2t\log n}(v_i) \cap L_i| > n,$$

where the last inequality follows by the fact $(1 + 1/t)^t \geq 2$ for any $t \geq 1$, and by the lemma's assumption that $|B_{2t\log n}(v_i) \cap L_i| \geq 1$. □

Lemma 5.2. *Assume $n \geq 2t\log n$. Then $\Pr(L_{n+1} \neq \emptyset) \leq 1/n$.*

Proof. We consider a node v and bound the probability of $v \in L_{n+1}$. Let $S = B_{2t\log n}(v)$, denote $k = |S|$, and let $v_{i_1}, \ldots, v_{i_k}$ be the vertices of S in lexicographic order. Since we assume $n \geq 2t\log n$, it is easy to see that $k \geq 2t\log n$. When processing each v_{i_j} for $j \in [k]$, if v is active in this step (i.e. $v \in L_{i_j}$), then by Lemma 5.1, v is still active at the end of the step (i.e. $v \in L_{i_j+1}$) with probability at most $1/2$. Therefore,

$$\Pr(v \in L_{n+1}) \leq \prod_{j=1}^{k} \Pr(v \in L_{i_j+1} \mid v \in L_{i_j}) \leq (1/2)^k \leq (1/2)^{2t\log n} \leq \frac{1}{n^2}.$$

and $\Pr(L_{n+1} \neq \emptyset) \leq 1/n$ follows by the union bound on all vertices $v \in V$. □

Next, we show that if $\mathcal{C}$ is a partition of V, then it is an $(O(t\log n), 1/t)$-TS partition.

Lemma 5.3. *If $L_{n+1} = \emptyset$, then $(\mathcal{C}, X)$ is a $(16t \log n, 1/t)$-TS partition.*

Proof. Recall that if C_i is added to $\mathcal{C}$, then it is non-empty, and that $C_i = B_{r_i}(v_i) \cap L_i$. Since $r_i \leq 8t \log n$, it follows that weak-diam$(C_i) \leq 16t \log n$. Next, we show that the cost ratio and two separation conditions hold. The cost ratio follows by:

$$\begin{aligned}
|X_i| &= |C_i \cap B_2(L_i \setminus C_i)| \\
&\leq |B_{r_i}(v_i) \cap L_i| - |B_{r_i-2}(v_i) \cap L_i| \\
&\leq (1 + \frac{1}{t}) \cdot |B_{r_i-2}(v_i) \cap L_i| - |B_{r_i-2}(v_i) \cap L_i| \\
&= \frac{|B_{r_i-2}(v_i) \cap L_i|}{t} \leq \frac{|C_i|}{t} \ .
\end{aligned}$$

We note that indeed $X_i = C_i \cap X$, since $X_i \subseteq C_i$ and the clusters are disjoint. Finally, we show the two separation property.

Assume for the sake of contradiction that the two separation property fails. Let $P_{u,v}$ be the shortest path between two vertices $u, v \notin X$ that belong to different clusters in $\mathcal{C}$ such that $P_{u,v}$ does not have two consecutive vertices from X. By minimality $\text{dist}_G(u, v) \leq 2$ (as otherwise we could have taken a shorter path). Let $C_i, C_j \in \mathcal{C}$ such that $u \in C_i$, $v \in C_j$, and suppose w.l.o.g. that $i < j$. As $\text{dist}_G(u, v) \leq |P_{u,v}| \leq 2$, it follows that $u \in C_i \cap B_2(L_j)$, and thus $u \in X$, a contradiction.

□

Combining Lemma 5.2 and Lemma 5.3, we immediately get the following as a corollary.

Corollary 5.4. *Assume $n \geq 2t \log n$. Then $(\mathcal{C}, X)$ is a $(16t \log n, 1/t)$-TS partition with probability at least $1 - (1/n)$.*

This concludes the description and analysis of $\mathcal{A}$. We call a randomness string R *good* w.r.t. graph G if by running $\mathcal{A}$ using randomness R, we have that the resulting clusters $(\mathcal{C}, X)$ are a $(16t \log n, 1/t)$-TS partition. In particular, Corollary 5.4 implies that for any graph G, at least a $1 - (1/n)$ fraction of the randomness strings are good w.r.t. G.

Encoding the randomness string of $\mathcal{A}$ and reducing its size: Let us consider the randomness used by $\mathcal{A}$, and its encoding. Naively, we can represent it as a choice of n even indices $r_i \in [2t \log n + 2, 8t \log n]$ for each $i \in [n]$. However, in our labeling scheme we run a sort of simulation of $\mathcal{A}$ in a distributed manner, where nodes have access to the shared randomness string. The naive representation does not lend itself well to that sort of distributed implementation, as it requires knowledge of the global ordering of node identifiers (i.e., knowing the lexicographic rank of each node u among all identifiers). Instead, we view the randomness as a function that assigns each possible identifier a random index, namely $R : [\text{poly}(n)] \to [2t \log n + 2, 8t \log n]$. With this representation, a node u can locally compute its random index from its identifier alone. The drawback is that describing R explicitly requires poly(n) random bits. We therefore apply a randomness reduction argument to reduce it to $O(\log n)$ bits, which we show later can be shared among all nodes using only $O(1)$ label bits.

Let $\mathcal{G}_n$ be the family of all graphs of size $|V(G)| = n$. Next, we prove using a standard argument that for graphs in $\mathcal{G}_n$ there exists a set of randomness strings $\{R_1, \ldots, R_{n^2}\}$ such that for each $G \in \mathcal{G}_n$ there exists an R_i that is a good randomness string for G.[12]

Lemma 5.5. *For any integer n, there exists a set $F_n : [n^2] \to \{0,1\}^*$ such that for any graph $G \in \mathcal{G}_n$ there exists an index $i \in [n^2]$ such that $F_n(i)$ is a good randomness string for G.*

Proof. We notice that algorithm $\mathcal{A}$ uses for graphs in $\mathcal{G}_n$ at most $O(n \log n)$ random bits. Choosing a randomness string R uniformly at random from $\{0,1\}^{O(n \log n)}$, we have that for a fixed $G \in \mathcal{G}_n$ that

$$\Pr(R \text{ is good randomness for } G) \geq 1 - (1/n).$$

Therefore, if we choose n^2 strings $R_1, \ldots, R_{n^2} \in \{0,1\}^{O(n \log n)}$ uniformly at random, we have that

$$\Pr(\exists_i R_i \text{ is good randomness for } G) \geq 1 - (1/n)^{n^2}.$$

By union bound over all graphs in $\mathcal{G}_n$ we have that

$$\Pr(\forall_{G \in \mathcal{G}_n} \exists_i R_i \text{ is good randomness for } G) \geq 1 - 2^{\binom{n}{2}} \cdot (1/n)^{n^2} > 0.$$

Therefore, there exists a set of n^2 randomness strings $R_1, \ldots, R_{n^2}$ such that for all $G \in \mathcal{G}_n$, there exists an index $i \in [n^2]$ such that R_i is a good randomness string for G. Setting for $i \in [n^2]$ the function $F_n(i) = R_i$, the claim follows. □

5.2 Labeling Scheme for General Graphs

Next, we show how to allow nodes to locally simulate $\mathcal{A}$ given $O(1)$-sized labels. Before showing the labeling scheme, we show for any $r \geq 1$ an $O(r)$-PLS that allows the honest prover to share a string S of size r across all nodes, with cost $O(1)$. Informally, we use this scheme to inform all nodes of an index of a good randomness string for G.

Lemma 5.6. *Let $r \geq 1$ be an integer. Then for any $S \in \{0,1\}^r$, there exists an $O(r)$-PLS $(\text{Prov}, \text{Ver})$ that uses $O(1)$ bits per node, such that for the honest prover, all nodes output S, and for any other labeling, either all nodes output some shared string S', or at least one node rejects.*

The proof of Lemma 5.6 is deferred to Section B, and we give a brief sketch of proof.

Proof sketch of Lemma 5.6. The key idea of the proof is to show the existence of a vertex set $U \subseteq V$ that splits the graph into connected components $C_1, \ldots, C_k$ of $G[V \setminus U]$ such that (a) $\forall_{i \in [k]} |C_i| = r$, and (b) for every $v \in V$ there exists $i \in [k]$ such that $\text{dist}_G(v, C_i) \leq r$.

Given this decomposition, we encode the string S on each of the connected components $C_1, \ldots, C_k$, assigning $O(1)$ bits of S to each node in the component. Therefore, every vertex v can retrieve S from its closest component C_i. Moreover, we can guarantee that, as long as G is connected, all nodes output the same string. To achieve this, we define an auxiliary graph G' whose vertices correspond to the clusters, and where two clusters are adjacent in G' if there exists a short path between them in G. We then observe that G' is connected, meaning that to ensure all nodes output the same string, it suffices to guarantee that neighboring clusters encode the same string. □

[12] In the literature, this type of argument is often referred to as "Newman's trick" [New91]. It was originally used in communication complexity to show that public coin protocols can be simulated by private coin protocols, and as a general method to reduce randomness.

We are now ready to prove Theorem 1.2. Let $(\text{Prov}_{\text{share}}, \text{Ver}_{\text{share}})$ be the labeling scheme for sharing strings described in Lemma 5.6. We construct an $O(t)$-PLS $(\text{Prov}_t, \text{Ver}_t)$ as follows.

Description of Prov_t: Let $(\mathcal{C}, X)$ be a (t, ε)-TS partition of G. For a randomness string $R : [\text{poly}(n)] \to [2t \log n + 2, 8t \log n]$, let T_R be the following set:

$$T_R = \{v_i \in V \mid C_i \text{ is taken into } \mathcal{C} \text{ when running } \mathcal{A} \text{ using randomness } R\}.$$

Recall that Lemma 5.5 provides a function $F_n : [n^2] \to \{0,1\}^*$, such that for any graph in $\mathcal{G}_n$ there exists an index $i \in [n^2]$ such that $F_n(i)$ is good randomness for it. Let $i_G \in [n^2]$ be a good randomness string for running $\mathcal{A}$ in G. Let $R^* = F_n(i_G)$, let $t(v) = 1$ if $v \in T_{R^*}$, and otherwise $t(v) = 0$, and let $x(v) = 1$ if $v \in X$, and otherwise $x(v) = 0$.

Denote $\text{Shared}(i_G, v) = \text{Prov}_{\text{share}}(G, i_G, v)$, the label given by $\text{Prov}_{\text{share}}$ to node v when sharing the binary encoding of i_G to all nodes. The prover gives the following label to each node $v \in V$.

$$\text{Prov}_t(G, I, v) = (\text{Shared}(i_G, v), t(v), x(v)) .$$

By Lemma 5.6, the cost of $\text{Prov}_{\text{share}}$ is $O(1)$ bits, and the size of both $t(v)$ and $x(v)$ is a single bit. Therefore in total, the cost of Prov_t is $O(1)$ bits.

Description of Ver_t: For convenience, we describe the verifier as a distributed algorithm from the viewpoint of v. We assume that the proof labels of any node v are given in a format encoding three strings $(\text{ShareLabel}(v), \text{InT}(v), \text{InX}(v))$, where $\text{InT}(v), \text{InX}(v)$ are single bits, and $\text{ShareLabel}(v)$ is a constant-sized string. Otherwise we have Ver_t reject.

Before describing the verifier, we define a simple sub-procedure called $\text{FindMyCluster}(v, R)$, running from the viewpoint of node v. Node v is given as input a function $R : [\text{poly}(n)] \to [2t \log n + 2, 8t \log n]$ mapping every possible identifier to an even index in that range. The sub-procedure returns the smallest identifier of a node $u \in B_{8t \log n}(v)$ satisfying both (a) $\text{InT}(u) = 1$ and (b) $\text{dist}_G(u, v) \leq R(\text{ID}(u))$. If there is no such node, then v outputs $\perp$. Informally, $\text{FindMyCluster}(v, R)$ simulates algorithm $\mathcal{A}$ from the viewpoint of v, assuming randomness string R, and assuming InT is the set of vertices for which a cluster was taken into $\mathcal{C}$. Its output is the identifier of v's cluster leader, where $\perp$ represents an error. We prove a few simple facts about FindMyCluster.

Lemma 5.7. *If for nodes $u, v \in V$ and some randomness string R we have* $\text{FindMyCluster}(u, R) = \text{FindMyCluster}(v, R) \neq \perp$, *then* $\text{dist}_G(u, v) \leq 16t \log n$.

Proof. Recall that FindMyCluster is a distributed algorithm that has access to a local neighborhood of radius $8t \log n$ and returns an identifier of some node in this neighborhood. Let w be a node such that $\text{ID}(w) = \text{FindMyCluster}(u, R) = \text{FindMyCluster}(v, R)$. Then necessarily $w \in B_{8t \log n}(u) \cap B_{8t \log n}(v)$, and by the triangle inequality we have $\text{dist}_G(u, v) \leq \text{dist}_G(u, w) + \text{dist}_G(w, v) \leq 16t \log n$. □

We next formalize the fact that FindMyCluster outputs the lexicographically smallest "taken" center whose ball contains the node.

Lemma 5.8. *Fix a randomness string R and labeling* $\mathrm{InT} : V \to \{0,1\}$. *For every node $u \in V$,*

$$\mathrm{FindMyCluster}(u,R) = \min\left\{\mathrm{ID}(w) \mid \ w \in B_{8t\log n}(u),\ \mathrm{InT}(w)=1,\ \mathrm{dist}_G(w,u) \le R(\mathrm{ID}(w))\right\},$$

where $\mathrm{FindMyCluster}(u,R) = \perp$ *if the set is empty.*

Proof. This follows by definition of FindMyCluster, which returns the smallest identifier among nodes satisfying the two conditions above. □

We are now ready to describe Ver_t:

(1) Node v runs $\mathrm{Ver}_{\mathrm{share}}$, using ShareLabel as the labeling and rejects if it rejects. Otherwise, node v obtains some string $i'(v)$, which it interprets as an encoding of a number. By the soundness of $(\mathrm{Prov}_{\mathrm{share}}, \mathrm{Ver}_{\mathrm{share}})$, we can assume that unless some node rejects, $i' = i'(v)$ is the same for all nodes, and omit v from the notation for convenience. Node v computes $F_n(i') = \widetilde{R}$ where $\widetilde{R} : [\mathrm{poly}(n)] \to [2t\log n + 2, 8t\log n]$.

(2) Node v rejects if $\mathrm{FindMyCluster}(v,\widetilde{R}) = \perp$.

(3) For every node $u \in B_{16t\log n}(v)$, node v runs $\mathrm{FindMyCluster}(u,\widetilde{R})$, and defines

$$\widetilde{C}_v(v) = \{u \in B_{16t\log n}(v) \mid \mathrm{FindMyCluster}(u,\widetilde{R}) = \mathrm{FindMyCluster}(v,\widetilde{R})\}.$$

Moreover, for any $w \in B_2(\widetilde{C}_v(v))$ it defines

$$\widetilde{C}_w(v) = \{u \in B_{16t\log n}(w) \mid \mathrm{FindMyCluster}(u,\widetilde{R}) = \mathrm{FindMyCluster}(w,\widetilde{R})\}.$$

(4) v adds $\widetilde{C}_w(v) \in \mathrm{Comp}(v)$ for every $w \in B_2(\widetilde{C}_v(v))$, and defines for each such $C \in \mathrm{Comp}(v)$ the set $X_C = \{u \in C \mid \mathrm{InX}(u) = 1\}$.

(5) Node v asserts that $\widetilde{C}_v(v)$ has weak diameter $\le 16t\log n$, satisfies the cost ratio property, and that any path from $\widetilde{C}_v(v) \setminus X_{\widetilde{C}_v(v)}$ to any node $u \in V \setminus \widetilde{C}_v(v)$ with $\mathrm{InX}(u) = 0$ passes through at least two consecutive nodes z_1, z_2 with $\mathrm{InX}(z_1) = \mathrm{InX}(z_2) = 1$.

Correctness: Assume every node v receives the label $\mathrm{Prov}_t(G,I,v)$. We prove that all nodes accept, and output their corresponding part in the TS partition $(\mathcal{C}, X)$ obtained by running algorithm $\mathcal{A}$ with randomness string R^*.

Lemma 5.9. *Let C_u be the cluster node u belongs to at the end of an execution of algorithm $\mathcal{A}$ with randomness string R^*. $\widetilde{C}_v(v) = C_v$, and for every $u \in B_2(C_v)$ we have $\widetilde{C}_u(v) = C_u$.*

Proof. Let $v_1, \ldots, v_n$ be the nodes of the graph, ordered lexicographically. Recall that

$$T_R = \{v_i \in V \mid C_i \text{ is taken into } \mathcal{C} \text{ when running } \mathcal{A} \text{ using randomness } R\},$$

and that $\mathrm{InT}(v) = 1$ if and only if $v \in T$. We prove by induction on $i \ge 1$ that if $v_i \in T$, then for all $u \in C_{v_i}$, we have $\mathrm{FindMyCluster}(u,R^*) = \mathrm{ID}(v_i)$. Let us consider the case of $i = 1$. If $v_1 \notin T$, then the claim holds trivially. Otherwise, the algorithm $\mathcal{A}$ sets $C_{v_1} = B_{R^*(v_1)}(v_1)$. Additionally, we have $\mathrm{InT}(v_1) = 1$, and that v_1 is the smallest identifier in the network. Therefore for a node $u \in V$

it holds that FindMyCluster$(u, R^*) = \mathrm{ID}(v_1)$ if and only if $u \in B_{R^*(v_1)}(v_1)$, which completes the basis case.

Next, we assume that the induction claim holds for some $i \in [n-1]$, and we prove the claim for $i+1$. By the induction assumption, we have that for all $j \leq i$ that for any $u \in C_{v_j}$ that FindMyCluster$(u, R^*) < \mathrm{ID}(v_{i+1})$. Therefore, the alive set satisfies

$$L_i = \{u \in V \mid \text{FindMyCluster}(u, R^*) \geq \mathrm{ID}(v_{i+1})\}.$$

Again, if $v_{i+1} \notin T_{R^*}$, then the claim trivially holds. Otherwise, we have that $C_{v_{i+1}}$ is taken into $\mathcal{C}$ in algorithm $\mathcal{A}$, and recall that $C_{v_{i+1}} = B_{R^*(v_{i+1})}(v_{i+1}) \cap L_i$.

On the other hand, by Lemma 5.8, any node $u \in L_i$ such that FindMyCluster$(u, R^*) \geq \mathrm{ID}(v_{i+1})$ is equivalent to the statement that u does not have a node $w \in B_{8t\log n}(u)$ such that $u \in B_{R^*(w)}(w)$, and that $\mathrm{InT}(w) = 1$. But since $\mathrm{InT}(v_{i+1}) = 1$, then for any $u \in B_{R^*(v_{i+1})}(v_{i+1})$ we have $v_{i+1} \in B_{8t\log n}(u)$, implying that FindMyCluster$(u, R^*) = \mathrm{ID}(v_{i+1})$. This concludes the proof by induction.

The lemma immediately follows, since $\widetilde{C}_v(v)$ every $\widetilde{C}_w(v)$ for each $w \in B_2(\widetilde{C}_v(v))$ are defined by FindMyCluster as

$$\widetilde{C}_v(v) = \{u \in B_{16t\log n}(v) \mid \text{FindMyCluster}(u, \widetilde{R}) = \text{FindMyCluster}(v, \widetilde{R})\},$$

and

$$\widetilde{C}_w(v) = \{u \in B_{16t\log n}(w) \mid \text{FindMyCluster}(u, \widetilde{R}) = \text{FindMyCluster}(w, \widetilde{R})\}.$$

In other words $\widetilde{C}_v(v) = C_v$, and $\widetilde{C}_w(v) = C_w$ for each $w \in B_2(C_v)$.

□

Recall that the prover set $\mathrm{InX}(u)$ if and only if $u \in X$, or in other words $X = \{u \in V \mid \mathrm{InX}(u) = 1\}$, therefore it follows that for any $C \in \mathrm{Comp}(v)$ we have $X_C(v) = C \cap X$. By the fact that R^* is a good random string, we are guaranteed that $(\mathcal{C}, X)$ is an $O((t\log n), 1/t)$-TS partition, and therefore all clusters satisfy the weak-diameter, cost ratio, and two separation properties. Therefore, all nodes accept, and correctness follows.

Soundness: Before proving soundness, we first prove a few simple claims. We define for every $w \in V$ the cluster

$$\widetilde{C}_w = \{u \in V \mid \text{FindMyCluster}(u, \widetilde{R}) = \text{FindMyCluster}(w, \widetilde{R})\}.$$

We define $\widetilde{\mathcal{C}} = \{\widetilde{C}_w \mid w \in V\}$. Additionally, define $\widetilde{X} = \{w \in V \mid \mathrm{InX}(w) = 1\}$ and for each $C \in \widetilde{\mathcal{C}}$ define $\widetilde{X}_C = C \cap \widetilde{X}$. Our overall goal is to show that the network outputs $(\widetilde{\mathcal{C}}, \widetilde{X})$ and that it is indeed an $(O(t\log n), 1/t)$-TS partition. We first show that $\widetilde{C}_w = \widetilde{C}_w(v)$ for any $v \in V$.

Lemma 5.10. *For any $v, w \in V$, we have $\widetilde{C}_w = \widetilde{C}_w(v)$ assuming $\widetilde{C}_w(v)$ is defined by v.*

Proof. Recall that

$$\widetilde{C}_w(v) = \{u \in B_{16t\log n}(w) \mid \text{FindMyCluster}(u, \widetilde{R}) = \text{FindMyCluster}(w, \widetilde{R})\}.$$

Notice that this definition is completely independent of v. Clearly $\widetilde{C}_w(v) \subseteq \widetilde{C}_w$. We show the other direction. Let $u \in \widetilde{C}_w$. Then FindMyCluster$(u, \widetilde{R}) =$ FindMyCluster$(w, \widetilde{R})$. By Lemma 5.7 we have $\mathrm{dist}_G(u, w) \leq 16t\log n$, i.e. $u \in \widetilde{C}_w(v)$ and the claim follows. □

The following follows as an immediate corollary.

Corollary 5.11. *Assume all nodes accept. Then for each $v \in V$, $\widetilde{C}_v \in \mathrm{Comp}(v)$ and for each $w \in B_2(\widetilde{C}_v)$ we have $\widetilde{C}_w \in \mathrm{Comp}(v)$. Moreover, for each $C \in \mathrm{Comp}(v)$ we have $X_C(v) = C \cap \widetilde{X}$.*

Proof. By Lemma 5.10 we have $\widetilde{C}_v(v) = \widetilde{C}_v$. Therefore $\widetilde{C}_v \in \mathrm{Comp}(v)$ and for each $w \in B_2(\widetilde{C}_v)$ we have $\widetilde{C}_w = \widetilde{C}_w(v) \in \mathrm{Comp}(v)$. The latter claim follows by definition of $\widetilde{X} = \{u \mid \mathrm{InX}(u) = 1\}$, and definition of $X_C(v) = \{u \in C \mid \mathrm{InX}(u) = 1\}$. □

Finally, we show that the pair $(\widetilde{\mathcal{C}}, \widetilde{X})$ is an $(O(t \log n), 1/t)$-TS partition of G

Lemma 5.12. *Assuming all nodes accept, the family of subsets $\widetilde{\mathcal{C}} = \{\widetilde{C}_u \mid u \in V\}$ is a partition of V. Moreover, $(\widetilde{\mathcal{C}}, \widetilde{X})$ is an $(O(t \log n), 1/t)$-TS partition of G.*

Proof. Let $u \in V$. Clearly $u \in \widetilde{C}_u$. We show that u appears in exactly one cluster. Indeed, assume that $u \in \widetilde{C}_w$ for some $w \in V$. Then by definition we have

$$\mathrm{FindMyCluster}(u, \widetilde{R}) = \mathrm{FindMyCluster}(w, \widetilde{R}).$$

And it follows that $\widetilde{C}_u = \widetilde{C}_w$. Finally, we show that $(\widetilde{\mathcal{C}}, \widetilde{X})$ is an $(O(t \log n), 1/t)$-TS partition. Since u does not reject, then by Step (2) of Ver_t we have $\mathrm{FindMyCluster}(u, \widetilde{R}) \neq \perp$, and by Step (5) we have that $\widetilde{C}_u(u)$ satisfies the low diameter, two separation, and cost ratio properties. □

We conclude that if all nodes accept, then $(\widetilde{\mathcal{C}}, \widetilde{X})$ is a $(O(t \log n), 1/t)$-TS partition of G, and each node v outputs $\mathrm{Comp}(v) = \{\widetilde{C}_u \mid u \in B_2(\widetilde{C}_v)\}$, with $X_C(v) = C \cap \widetilde{X}$ for every $C \in \mathrm{Comp}(v)$, and soundness follows. By combining this proof labeling scheme with Lemma 3.2, we obtain Theorem 1.2.

Discussion on knowledge of n: In this section, we assume the verifier has knowledge of n when locally constructing the function F_n of Lemma 5.5, that is used to obtain our randomness strings. However, we believe this assumption can be removed with relative ease by using the string sharing PLS of Lemma 5.6 to share the value n' in addition to the index i'. We note that while n' might not be the real value of n, we are still guaranteed that the generated randomness string is identical among all nodes. Moreover, since the nodes verify the properties of the resulting TS partition, if they all accept, then the TS partition is valid even if n' is not the real size of the graph. Once the assumption of knowing n is removed, we can also remove the assumption that the graph is connected, since we can act on each component separately.

6 Refuting Stronger Variants of the Tradeoff Conjecture

In this section, we refute a stronger variant of the Tradeoff Conjecture, and show a problem where every node has $|B_t(v)| = \Omega(n)$, but we cannot get an improvement of better than $O(1/t)$ when considering a t-PLS compared to a 1-PLS.

Theorem 1.6. *There exists a predicate P on an infinite family of planar graphs $\mathcal{G} = \{G_{t,m}\}_{t,m=1}^{\infty}$ and on some auxiliary input set $\mathcal{I}$, where for $G_{t,m} \in \mathcal{G}$ we have $\min_{v \in V(G_{t,m})} |B_t(v)| = \Omega(|V(G_{t,m})|)$, and there is a 1-PLS for P with cost $O(m)$, but any t-PLS for P has cost $\Omega(m/t)$.*

In our proof, we use a reduction to 2-party non-deterministic communication complexity. We begin with a brief overview of the 2-party non-deterministic communication model and the results relevant for our reduction. For simplicity, instead of describing the standard non-deterministic communication model, we describe a slight (but standard) variant. This variant is equivalent to the standard model up to an additive $O(1)$ term in their respective complexity measures.

Non-deterministic communication complexity: For our purposes, the non-deterministic 2-party communication model consists of two players, Alice and Bob, with private inputs $X \in \mathcal{X}$ and $Y \in \mathcal{Y}$, respectively, for some input domain $\mathcal{X} \times \mathcal{Y}$. A protocol with cost p in this setting is defined as follows. Initially, both Alice and Bob are given a shared string $W \in \{0,1\}^{\leq p}$, called the *witness string*. Without communicating, both Alice and Bob output either *accept* or *reject* based solely on their private input and the witness W. For a predicate $P : \mathcal{X} \times \mathcal{Y} \to \{0,1\}$, we say that a non-deterministic p-cost protocol solves P if:

- **Correctness:** For any $(X,Y) \in \mathcal{X} \times \mathcal{Y}$ such that $P(X,Y) = 1$, there exists a witness string W with $|W| \leq p$ for which both players accept.
- **Soundness:** For any $(X,Y) \in \mathcal{X} \times \mathcal{Y}$ such that $P(X,Y) = 0$, and for every witness string W with $|W| \leq p$, at least one of Alice or Bob rejects.

We briefly remark that in the standard model, nodes are allowed to communicate, and the cost is the size of W plus the total number of bits communicated. Though, using a standard argument, the difference in cost between the two models is at most two bits. Informally, this is because instead of having lengthy communication, we can always encode the transcript inside W and have both nodes verify the transcript using a single bit each (See [KN97] for an overview).

Finally, we describe the equality function and the lower bound needed for our results. The equality function $\mathrm{EQ}_k : \{0,1\}^k \times \{0,1\}^k \to \{0,1\}$ is defined by $\mathrm{EQ}_k(X,Y) = 1$ if and only if $X = Y$.

Lemma 6.1 ([KN97] Chapter 2.1)**.** *Any non-deterministic 2-player protocol solving* EQ_k *has cost* $\Omega(k)$.

Proof of Theorem 1.6: Next, we prove Theorem 1.6. We begin by describing $G_{t,m}$ and P, and continue to show the existence of an $O(m)$ cost 1-PLS for P, and the lower bound of $\Omega(m/t)$ on the cost of any t-PLS for P. The graph $G_{t,m}$ is defined as follows for integer $t \geq 1$, and odd integer $m \geq 3$. For $i \in [2t+3]$, we define the set of vertices,

$$V_i = \begin{cases} \{v^i\}, & i \text{ is odd}, \\ \{v^i_1, \ldots, v^i_m\}, & i \text{ is even}. \end{cases}$$

Let $V = \bigcup_{i \in [2t+3]} V_i$ be a union of the vertex sets, and note that $|V| = \Theta(mt)$. Let $G_{t,m} = (V,E)$ be a graph where $E = \{\{u,w\} \mid i \in [2t+2], u \in V_i, w \in V_{i+1}\}$, i.e. $G_{t,m}$ contains for every $i \in [2t+2]$ all edges between V_i and V_{i+1} (See Figure 2).

We define the problem P as follows. In this problem, both nodes $v^1 \in V_1, v^{2t+3} \in V_{2t+3}$ receive an input $I(u) \in \{0,1\}^{m^2}$. We ignore all additional inputs on nodes not in $V_1 \cup V_{2t+3}$, i.e. for nodes with even degree. We define P so that $(G_{t,m}, I) \in P$ if and only if $I(v^1) = I(v^{2t+3})$. Denote $\deg(v)$ as the degree of a vertex v. We recall that m is an odd integer, and note the following trivial properties.

Lemma 6.2. *The following holds for any $v \in V$.*

1. *If $\deg(v) = 2$, then v is at an even layer.*
2. *If $\deg(v) > 2$ and is odd, then $v \in V_1 \cup V_{2t+3}$. Moreover, $\deg(v) = m$.*
3. *If $\deg(v) > 2$ and is even, then v is at an odd layer, but not in $V_1 \cup V_{2t+3}$. Moreover, $\deg(v) = 2m$.*

Lemma 6.3. *There exists a 1-PLS for P in $G_{t,m}$ with cost $O(m)$.*

Proof. We describe a 1-PLS for P, denoted $(\text{Prov}_1, \text{Ver}_1)$, with cost $O(m)$.

Description of Prov_1 Let $I_1, \ldots, I_m$ be a partition of $I(v^1)$ into m equal-sized segments. For any $j \in [2t+3]$, if j is odd, then $\text{Prov}_1(G, I, v) = \phi$, i.e. an empty label. If j is even, then for $i \in [m]$, node v_i^j is given the label we set $\text{Prov}_1(G, I, v_i^j) = (i, I_i)$. We remark that indeed all label sizes are of size at most $O(m)$, hence the cost is at most $O(m)$.

Description of Ver_1 We define Ver_1 as a distributed protocol from the viewpoint of v:

1. If $\deg(v) = 2$ (i.e. v is at an even layer), v accepts.
2. If $\deg(v) > 2$ and is even (i.e. v is at an odd layer, but not in $V_1 \cup V_{2t+3}$), then v sets $m = \deg(v)/2$, and asserts that for each $i \in [m]$ there are exactly two labels of neighbors (i_1, S_1) and (i_2, S_2) such that $i = i_1 = i_2$ and $S_1 = S_2$.
3. If $\deg(v) > 2$ and is odd (i.e. $v \in V_1 \cup V_{2t+3}$), v asserts that for each $i \in [m]$ there is exactly one label of neighbors (i_1, S_1) where $i = i_1$, and that its input is equal to the concatenation of strings $I(v) = S_1 \circ \cdots \circ S_m$.

Correctness We split into cases by layer. Every odd layer vertex always accepts. For each of the layers V_1, V_{2t+3} we see that indeed for their unique node v, they have m neighbors, each with label (i, I_i), and indeed $I(v) = I_1 \circ \cdots \circ I_m$. Similarly, for the rest of the odd layers, each node receives for each $i \in [m]$ the label (i, I_i) exactly twice from its neighbors, and we conclude that all nodes accept.

Soundness Assume all nodes accept. We prove by induction over all even integers in $j \in [2t+3]$ that for all $i \in [m]$ there is exactly one node $u_{i,j} \in V_j$ that has the label (i, I_i). For the case $j = 2$ this must hold, otherwise v^1 rejects. Assume this for even layer $j \geq 2$, therefore layer $j+2$ must also have exactly one node $u_{i,j+2} \in V_{j+2}$ that has the label (i, I_i), otherwise v^{j+1} rejects. Finally, we must have $I(v^{2t+3}) = I_1 \circ \cdots \circ I_m$, otherwise v^{2t+3} rejects. We conclude that $I(v^1) = I(v^{2t+3})$. □

Finally, we show that any t-PLS certifying P in $G_{t,m}$ has cost $\Omega(m/t)$, which is shown by reduction to non-deterministic 2-player equality.

Lemma 6.4. *Any t-PLS certifying P in $G_{t,m}$ has cost $\Omega(m/t)$.*

Proof. Let $\Pi_t = (\text{Prov}_t, \text{Ver}_t)$ be a t-PLS for certifying P in $G_{t,m}$ with cost p. We show that this implies a non-deterministic 2-player protocol for (m^2)-bit equality with cost $O(p \cdot m \cdot t)$. Combining with the $\Omega(m^2)$ complexity lower bound on EQ_{m^2} (Lemma 6.1), we obtain by rearranging that $p = \Omega(m/t)$.

We describe a 2-player protocol that uses Π_t to obtain the complexity mentioned above. Let X, Y be the private inputs of Alice and Bob respectively. Alice and Bob each construct the graph $G_{t,m}$. Alice assigns X to be the input of v^1, and similarly, Bob assigns Y to be the input of v^{2t+3}. Both players interpret $W = (\ell_1, \ldots, \ell_{|V(G)|})$ as a labeling on the nodes V (rejecting if W does not encode $|V(G)|$ labels). Finally, Alice checks whether all nodes in $\bigcup_{i=1}^{t+1} V_i$ accept, and Bob checks whether all nodes of $\bigcup_{i=t+2}^{2t+3} V_i$ accept. If so, they accept, and otherwise reject.

We analyze the correctness of the protocol above. Assuming $X = Y$, we show there exists a witness string W causing both players to accept. Indeed, taking $W = (\text{Prov}_t(G, I, v))_{v \in V}$, and noting that $X = Y$ implies $I(v^1) = I(v^{2t+3})$, then by correctness of Π_t all nodes accept. Otherwise, assume that $X \neq Y$, i.e. $I(v^1) \neq I(v^{2t+3})$. Therefore, by the soundness of Π_t, for any labeling $\ell : V \to \{0,1\}^*$ the verifier Ver_t rejects on at least one node v. We note that either $v \in \bigcup_{i=1}^{t+1} V_i$ or $v \in \bigcup_{i=t+2}^{2t+3} V_i$. In the first case, Alice rejects, and in the latter Bob rejects. □

Acknowledgments

This work is supported in part by the Israel Science Foundation, grant No. 1042/22 and 800/22.

References

[ACF+23] Virginia Ardévol Martínez, Marco Caoduro, Laurent Feuilloley, Jonathan Narboni, Pegah Pournajafi, and Jean-Florent Raymond. A lower bound for constant-size local certification. *Theoretical Computer Science*, 971:114068, 2023. 1

[Bar96] Yair Bartal. Probabilistic approximations of metric spaces and its algorithmic applications. In *37th Annual Symposium on Foundations of Computer Science, FOCS '96, Burlington, Vermont, USA, 14-16 October, 1996*, pages 184–193, 1996. 6, 7, 18

[BFP24] Nicolas Bousquet, Laurent Feuilloley, and Théo Pierron. Local certification of graph decompositions and applications to minor-free classes. *Journal of Parallel and Distributed Computing*, 193:104954, 2024. 2

[BFZ25] Nicolas Bousquet, Laurent Feuilloley, and Sébastien Zeitoun. Local certification of local properties: Tight bounds, trade-offs, and new parameters. *SIAM J. Discret. Math.*, 39(4):1974–2002, 2025. 1

[CF25] Jonathan Conroy and Arnold Filtser. How to protect yourself from threatening skeletons: Optimal padded decompositions for minor-free graphs. In Michal Koucký and Nikhil Bansal, editors, *Proceedings of the 57th Annual ACM Symposium on Theory of Computing, STOC 2025, Prague, Czechia, June 23-27, 2025*, pages 2281–2292. ACM, 2025. 6, 16

[CHPP20] Keren Censor-Hillel, Ami Paz, and Mor Perry. Approximate proof-labeling schemes. *Theoretical Computer Science*, 811:112–124, 2020. 2

[CKM25] Linda Cook, Eun Jung Kim, and Tomáš Masařík. A tight meta-theorem for local certification of mso2 properties within bounded treewidth graphs. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*, pages 110–120, 2025. 2

[FBP22] Laurent Feuilloley, Nicolas Bousquet, and Théo Pierron. What can be certified compactly? compact local certification of mso properties in tree-like graphs. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing*, page 131–140, New York, NY, USA, 2022. Association for Computing Machinery. 2

[Feu21] Laurent Feuilloley. Introduction to local certification. *Discret. Math. Theor. Comput. Sci.*, 23(3), 2021. 1, 2

[FF16] Laurent Feuilloley and Pierre Fraigniaud. Survey of distributed decision. *Bull. EATCS*, 119, 2016. 2

[FFH+21] Laurent Feuilloley, Pierre Fraigniaud, Juho Hirvonen, Ami Paz, and Mor Perry. Redundancy in distributed proofs. *Distributed Comput.*, 34(2):113–132, 2021. 1, 2, 3, 7, 9

[FFM+21] Laurent Feuilloley, Pierre Fraigniaud, Pedro Montealegre, Ivan Rapaport, Éric Rémila, and Ioan Todinca. Compact distributed certification of planar graphs. *Algorithmica*, 83(7):2215–2244, 2021. 2, 5

[FJKS24] Laurent Feuilloley, Jan Janoušek, Jan Matyáš Křišťan, and Josef Erik Sedláček. Decreasing verification radius in local certification. In *International Symposium on Algorithmics of Wireless Networks*, pages 188–201. Springer, 2024. 1

[FMRT24] Pierre Fraigniaud, Pedro Montealegre, Ivan Rapaport, and Ioan Todinca. A meta-theorem for distributed certification. *Algorithmica*, 86(2):585–612, 2024. 2

[FOS21] Orr Fischer, Rotem Oshman, and Dana Shamir. Explicit space-time tradeoffs for proof labeling schemes in graphs with small separators. In *25th International Conference on Principles of Distributed Systems (OPODIS)*, volume 217 of *LIPIcs*, pages 21:1–21:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. 1, 2, 3, 7, 17

[GS16] Mika Göös and Jukka Suomela. Locally checkable proofs in distributed computing. *Theory Comput.*, 12(1):1–33, 2016. 2, 5

[KK07] Amos Korman and Shay Kutten. Distributed verification of minimum spanning trees. *Distributed Comput.*, 20(4):253–266, 2007. 2, 5, 9

[KKP10] Amos Korman, Shay Kutten, and David Peleg. Proof labeling schemes. *Distributed Comput.*, 22(4):215–233, 2010. 1, 2, 5

[KN97] Eyal Kushilevitz and Noam Nisan. *Communication complexity*. Cambridge University Press, 1997. 26

[New91] Ilan Newman. Private vs. common random bits in communication complexity. *Information Processing Letters*, 39(2):67–71, 1991. 21

[OPR17] Rafail Ostrovsky, Mor Perry, and Will Rosenbaum. Space-time tradeoffs for distributed verification. In *International Colloquium on Structural Information and Communication Complexity*, pages 53–70. Springer, 2017. 1, 7

[Suo13] Jukka Suomela. Survey of local algorithms. *ACM Comput. Surv.*, 45(2):24:1–24:40, 2013. 2

A Missing Proofs from Section 4

In this section, we prove Lemma 4.5. Namely, we show that any (t,ε)-TS partition can be certified and outputted in any graph using a $O(t)$-PLS with cost $O(\log n)$.

Lemma 4.5. *In any graph family $\mathcal{G}$ and any $t(n) \geq 1, \varepsilon(n) \in [0,1)$ there exists a $O(t)$-PLS that certifies and outputs a $(t(n),\varepsilon(n))$-TS partition with cost $O(\log n)$.*

Let $r = 3t+2$. We construct an r-PLS $(\text{Prov}, \text{Ver})$ certifying and outputting a (t,ε)-TS partition as follows.

Description of Prov**:** Let $(\mathcal{C}, X)$ be a (t,ε)-TS partition of G. For each cluster $C \in \mathcal{C}$, the honest prover assigns a unique identifier $\text{ID}(C) \in [n]$, and gives node v the label

$$\text{Prov}(G, I, v) = (\text{ID}(C_v), x(v)),$$

where $x(v)$ is a single bit set to $x(v) = 1$ if $v \in X$, and $x(v) = 0$ otherwise. We note that the size of the label for each node is $O(\log n)$ bits.

Description of Ver**:** For convenience, we describe Ver as a distributed algorithm running from the viewpoint of node v. We assume that v receives for every node $u \in B_r(v)$ a label $(\text{CompID}(u), \text{InX}(u))$, where $\text{InX}(u)$ is a single bit (otherwise v rejects). Given the r-hop topology of v, node v defines

$$\widetilde{C}_v(v) = \{w \in B_r(v) \mid \text{CompID}(v) = \text{CompID}(w)\},$$

and for each $u \in B_2(\widetilde{C}_v(v))$ defines the cluster

$$\widetilde{C}_u(v) = \{w \in B_r(u) \mid \text{CompID}(u) = \text{CompID}(w)\}.$$

It then defines $\text{Comp}(v) = \{\widetilde{C}_u(v) \mid u \in B_2(\widetilde{C}_v(v))\}$, and for each $C \in \text{Comp}(v)$ defines the set $X_C(v) = \{w \in C \mid \text{InX}(w) = 1\}$.

Node v asserts the following conditions, and rejects if at least one assertion fails.

(1) Node v asserts that for each $C \in \text{Comp}(v)$ that weak-diam$(C) \leq t$, and the cost ratio is at most $|X_C(v)|/|C| \leq \varepsilon$.

(2) Node v asserts that there are no two nodes $u \in \widetilde{C}_v(v)$ and $w \notin \widetilde{C}_v(v)$, with $\text{InX}(u) = \text{InX}(w) = 0$, such that $\text{dist}_G(u,w) \leq 2$.

If all assertions are true, then Ver accepts and outputs $\text{Comp}(v)$, and for each $C \in \text{Comp}(v)$ outputs the set $X_C(v)$ as its corresponding X-set. This concludes the description of the verifier.

Correctness: Since $(\mathcal{C}, X)$ is a (t, ε)-TS partition, and each $C \in \mathcal{C}$ is given a distinct identifier, then $\widetilde{C}_v(v) = C_v$, and for any $u \in B_2(C_v)$, it similarly holds that $\widetilde{C}_u(v) = C_u$. Moreover, for each $u \in B_2(C_v)$, the cluster $\widetilde{C}_u(v) = C_u$ satisfies the weak-diameter, two separation, and cost ratio conditions. Therefore, all assertions of Ver running on v are true, and it accepts and outputs $\mathrm{Comp}(v) = \{C_u \mid u \in B_2(C_v)\}$, and for each $C \in \mathrm{Comp}(v)$ the set $X_C(v) = X_C$.

Soundness: Assume that for every node $v \in V$, Ver accepts for some labeling ℓ. We begin by showing a few simple claims.

Lemma A.1. *If $u \in B_r(v)$, and $u \notin \widetilde{C}_v(v)$, then* $\mathrm{CompID}(u) \neq \mathrm{CompID}(v)$.

Proof. By definition, $\widetilde{C}_v(v) = \{w \in B_r(v) \mid \mathrm{CompID}(w) = \mathrm{CompID}(v)\}$. Hence, if $u \in B_r(v)$ and $u \notin \widetilde{C}_v(v)$, then necessarily $\mathrm{CompID}(u) \neq \mathrm{CompID}(v)$. □

Lemma A.2. *Assume all nodes accept. For any $u, v, w \in V$, if $\widetilde{C}_w(u) \in \mathrm{Comp}(u)$ and $\widetilde{C}_w(v) \in \mathrm{Comp}(v)$, then $\widetilde{C}_w(u) = \widetilde{C}_w(v)$. Moreover, in that case $X_{\widetilde{C}_w}(u) = X_{\widetilde{C}_w}(v)$.*

Proof. Assume by contradiction that there exists a node $z \in \widetilde{C}_w(v) \setminus \widetilde{C}_w(u)$. Since $z \in \widetilde{C}_w(v)$, by definition we have $z \in B_r(w)$ and $\mathrm{CompID}(z) = \mathrm{CompID}(w)$. On the other hand, since $z \notin \widetilde{C}_w(u)$ while $\mathrm{CompID}(z) = \mathrm{CompID}(w)$, again by definition of $\widetilde{C}_w(u)$ it must be that $z \notin B_r(w)$, a contradiction. Therefore $\widetilde{C}_w(v) = \widetilde{C}_w(u)$.

Moreover, the corresponding sets $X_{\widetilde{C}_w}(u)$ and $X_{\widetilde{C}_w}(v)$, defined as the set of vertices $y \in \widetilde{C}_w$ with $\mathrm{InX}(y) = 1$, are equal as well. □

Lemma A.2 shows that given a labeling ℓ where all nodes accept, the clusters $\widetilde{C}_u(v)$ are consistent whenever they are defined. In the remainder of the section, we denote $\widetilde{C}_u = \widetilde{C}_u(u)$, and $X_{\widetilde{C}_u} = X_{\widetilde{C}_u}(u)$. We note that this is defined for all $u \in V$, since we always have $\widetilde{C}_u(u) \in \mathrm{Comp}(u)$. We let $\widetilde{\mathcal{C}} = \{\widetilde{C}_u \mid u \in V\}$, and $\widetilde{X} = \bigcup X_{\widetilde{C}_u}$. We show that $(\widetilde{\mathcal{C}}, \widetilde{X})$ is an (t, ε)-TS partition.

Lemma A.3. *Assuming all nodes accept, the family of subsets $\widetilde{\mathcal{C}} = \{\widetilde{C}_u \mid u \in V\}$ is a partition of V. Moreover, $(\widetilde{\mathcal{C}}, \widetilde{X})$ is an (t, ε)-TS partition of G.*

Proof. To show that $\widetilde{\mathcal{C}}$ is a partition, we show that every node appears in some cluster, and that any two clusters that the clusters are disjoint. The fact that every $u \in V$ appears in some cluster follows from $u \in \widetilde{C}_u$ and $\widetilde{C}_u \in \widetilde{\mathcal{C}}$.

Assume that $z \in \widetilde{C}_u \cap \widetilde{C}_v$. Then $\mathrm{CompID}(u) = \mathrm{CompID}(z) = \mathrm{CompID}(v)$. Since $\widetilde{C}_u \in \mathrm{Comp}(u)$ and $\widetilde{C}_v \in \mathrm{Comp}(v)$, both u and v accept Assertion (1), and hence $\text{weak-diam}(\widetilde{C}_u) \leq t$ and $\text{weak-diam}(\widetilde{C}_v) \leq t$. It follows that $\mathrm{dist}_G(u, v) \leq 2t$. We claim that $\widetilde{C}_u = \widetilde{C}_v$. Indeed, let $y \in \widetilde{C}_u$. Then $\mathrm{CompID}(y) = \mathrm{CompID}(u) = \mathrm{CompID}(v)$. Moreover, since $\text{weak-diam}(\widetilde{C}_u) \leq t$, we have $\mathrm{dist}_G(u, y) \leq t$, and thus by the triangle inequality $\mathrm{dist}_G(v, y) \leq \mathrm{dist}_G(v, u) + \mathrm{dist}_G(u, y) \leq 2t + t = 3t \leq r$. Therefore $y \in B_r(v)$ and has the same component identifier as v, i.e., $y \in \widetilde{C}_v$. The symmetric argument shows $\widetilde{C}_v \subseteq \widetilde{C}_u$, concluding $\widetilde{C}_u = \widetilde{C}_v$.

We conclude that since every node passes Assertions (1),(2), then all clusters $\widetilde{C}_v$ have the weak diameter, cost ratio, and two separation properties. In other words, the pair $(\widetilde{\mathcal{C}}, \widetilde{X})$ is a (t, ε)-TS partition. □

Finally, we show that all nodes output their respective local part of $(\widetilde{\mathcal{C}}, \widetilde{X})$, and Lemma 4.5 follows.

Lemma A.4. *Assume all nodes accept. Then for each $v \in V$, $\widetilde{C}_v \in \mathrm{Comp}(v)$ and for each $u \in B_2(\widetilde{C}_v)$ we have $\widetilde{C}_u \in \mathrm{Comp}(v)$. Moreover, for each $C \in \mathrm{Comp}(v)$ we have $X_C(v) = C \cap \widetilde{X}$.*

Proof. By definition $\widetilde{C}_v = \widetilde{C}_v(v)$ and indeed is defined by v and added to $\mathrm{Comp}(v)$. Therefore, $B_2(\widetilde{C}_v) = B_2(\widetilde{C}_v(v))$. In particular, v defines for any $u \in B_2(\widetilde{C}_v)$ a cluster $\widetilde{C}_u(v)$ and adds it to $\mathrm{Comp}(v)$. By Lemma A.2, we have $\widetilde{C}_u(v) = \widetilde{C}_u$, and the first part of the claim follows.

For latter claim, let $C \in \mathrm{Comp}(v)$. By definition of $\mathrm{Comp}(v)$, there exists $u \in B_2(\widetilde{C}_v)$ such that $C = \widetilde{C}_u(v)$. By Lemma A.2, $C = \widetilde{C}_u$ and moreover $X_C(v) = X_{\widetilde{C}_u}$. Since $\widetilde{\mathcal{C}}$ is a partition of V (Lemma A.3), we have $X_{\widetilde{C}_u} = \widetilde{C}_u \cap \widetilde{X} = C \cap \widetilde{X}$. □

B Missing Proofs from Section 5

In this section, we prove Lemma 5.6. We start by first proving the following decomposition lemma.

Lemma B.1. *Let $r \geq 1$ be an integer parameter, and let $G = (V, E)$ be a connected graph with $|V| > r$. There exists a vertex set $U \subseteq V$ that satisfies the following properties. Let $C_1, \ldots, C_k$ be the connected components of $G[V \setminus U]$. Then, (a) For every $i \in [k]$, $|C_i| = r$, and (b) For every $v \in V$, there exists $i \in [k]$ such that $v \in B_r(C_i)$.*

Proof. We use the following simple iterative procedure to construct the set U and clusters $C_1, \ldots, C_k$, proving this decomposition's existence. At each step we have our set of active nodes V_i, initially set at $V_1 = V$. At the i-th step, if there is no connected component remaining in the graph with size at least r, we terminate the process. Otherwise, set C_i to be an arbitrary connected subgraph of such a component, of size exactly r, and let

$$U_i = \{u \in V_i \setminus C_i \mid u \in B_1(C_i)\}.$$

We set $V_{i+1} = V_i \setminus (C_i \cup U_i)$. Assuming the process ended at step $k+1$, we set $U = V_{k+1} \cup \bigcup_{j=1}^{k} U_j$.

Next we prove correctness of this process.

- Property (a) follows trivially from the fact that each cluster is taken to be a connected subgraph of size exactly r.
- For Property (b), we consider a few simple cases. If $v \in C_i \cup U_i$ for some $i \in [k]$, then $v \in B_1(C_i)$. Otherwise, $v \in V_{k+1}$, i.e. v is one of the nodes remaining at the end of the process. Since $k+1$ is the last step, the connected component of v in V_{k+1} is of size at most $r-1$. In particular, $\mathrm{dist}_G(v, \cup_{i=1}^{k} U_i) \leq r-1$, implying $v \in B_r(C_i)$ for some $i \in [k]$.

□

Next, we show an implication of Property Lemma B.1(b), which we use for the soundness of the protocol. For an integer parameter $r \geq 1$, and set $U \subseteq V$, define the graph $G'(r, U)$ as follows. Let $C_1, \ldots, C_k$ be the maximal connected components of $G[V \setminus U]$. Let $G'(r, U)$ be the graph with vertices $V' = \{v'_1, \ldots, v'_k\}$, such that two vertices $v'_i, v'_j \in V'$ are connected by an edge if and only if $\mathrm{dist}_G(C_i, C_j) \leq 2r + 1$.

Lemma B.2. *Let $r \geq 1$ be an integer parameter, let $G = (V, E)$ be a connected graph, and let $U \subseteq V$. If $\max_{v \in V} \mathrm{dist}_G(v, V \setminus U) \leq r$, then $G'(r, U)$ is connected.*

Proof. Consider a path $p = (w_1 = u, w_2, w_3, \ldots, w_\ell = v)$ between two nodes $u, v \in V \setminus U$. For $j \in [\ell]$, let C_{i_j} denote the maximal connected component in $G[V \setminus U]$ closest to w_j in G (breaking ties arbitrarily). By assumption, $\mathrm{dist}_G(w_j, C_{i_j}) \leq r$. It follows by the triangle inequality that $\mathrm{dist}_G(C_{i_j}, C_{i_{j+1}}) \leq 2r+1$, i.e. v'_{i_j} is connected by an edge to $v'_{i_{j+1}}$ in $G'(r, U)$, for all $1 \leq j \leq \ell - 1$. Hence, v'_i, v'_j are connected in $G'(r, U)$ if there is a path between the connected components C_i and C_j in G. Since G is connected, we conclude that $G'(r, U)$ is also connected. □

Before the proof of Lemma 5.6, recall the functions Lex and DecodeLex for encoding and decoding strings across a cluster C that were introduced in Section 3. In short, $\mathrm{Lex}(C, S, v)$ pads S to a multiple of $|C|$, and returns the i-th block, where i is the lexicographic rank of v in C. Conversely, $\mathrm{DecodeLex}(C, \ell)$, where $\ell(v) = \mathrm{Lex}(C, S, v)$ for $v \in C$, reconstructs the original string S by concatenating these blocks in order and removing the padding.

Proof of Lemma 5.6 We show a $(4r+2)$-PLS for the problem described. We assume that the labeling given to a node v is of the form $(\mathrm{InU}(v), \mathrm{PartS}(v))$, where $\mathrm{InU}(v) \in \{0, 1\}$ is one bit and $\mathrm{PartS}(v) \in \{0, 1\}^*$ is an arbitrary string (otherwise, the verifier rejects). Given a labeling $(\mathrm{InU}, \mathrm{PartS})$ on all nodes, let $V_0 = \{v \in V \mid \mathrm{InU}(v) = 0\}$. A 0-cluster $C \subseteq V$ is a maximal connected component in $G[V_0]$, i.e. a maximal connected component in the graph induced by nodes with $\mathrm{InU}(v) = 0$.

Description of Prov**:** Fix a set $U \subseteq V$ satisfying the properties described in Lemma B.1. Set Prov as a function which gives labels $(\mathrm{InU}(v), \mathrm{PartS}(v))$ to each node v, where (a) Set $\mathrm{InU}(v) = 1$ if $v \in U$, and $\mathrm{InU}(v) = 0$ otherwise. (b) If $v \in U$, set $\mathrm{PartS}(v) = \emptyset$. Otherwise, let C_v be the 0-cluster that contains v, and set $\mathrm{PartS}(v) = \mathrm{Lex}(C_v, S, v)$.

Description of Ver**:** We describe the verifier from the viewpoint of v, and say that node v rejects if $\mathrm{Ver}(G, I, v) = \text{reject}$, and node v accepts otherwise. First, v rejects if its label is not in the format $(\mathrm{InU}(v), \mathrm{PartS}(v))$ with $\mathrm{InU}(v) \in \{0, 1\}$. Let $\mathrm{SeenClusters}(v) = \{C \subseteq B_{4r+1}(v) \mid C \text{ is a 0-cluster}\}$. Node v rejects if $|\mathrm{SeenClusters}(v)| = 0$, if $\min_{C \in \mathrm{SeenClusters}(v)} \mathrm{dist}_G(v, C) > r$, or if there is a 0-cluster C such that $\mathrm{dist}_G(v, C) \leq r$, but such that $|C| \neq r$.

Then v runs for all $C \in \mathrm{SeenClusters}(v)$ the function $\mathrm{DecodeLex}(C, \mathrm{PartS})$ and obtains a string $S'(C)$. It then asserts that for all $C, C' \in \mathrm{SeenClusters}(v)$ we have $S'(C) = S'(C')$. If this assertion is false, v rejects. Otherwise, there is some string $S' = S'(C)$ for all 0-clusters $C \in \mathrm{SeenClusters}(v)$. v accepts and outputs the string S'.

Correctness: We need to show that $\mathrm{Ver}(G, I, \mathrm{Prov}(G, I), v)$ accepts for all $v \in V$, and that all nodes output the string S. By Lemma B.1(b) there exists a 0-cluster C such that $\mathrm{dist}_G(v, C) \leq r$, and in particular $C \subseteq B_{4r+1}(v)$, so $|\mathrm{SeenClusters}(v)| \geq 1$ and $\min_{C' \in \mathrm{SeenClusters}(v)} \mathrm{dist}_G(v, C') \leq r$. Let $C \in \mathrm{SeenClusters}(v)$. We note that for any $u \in C$ we have $\mathrm{PartS}(u) = \mathrm{Lex}(C, S, u)$. Therefore, $\mathrm{DecodeLex}(C, \mathrm{PartS}) = S$, i.e. $S'(C) = S$ for all $C \in \mathrm{SeenClusters}(v)$. Correctness follows.

Soundness: Assume that all nodes $v \in V$ accept on some labeling $\ell = (\mathrm{InU}, \mathrm{PartS})$. Let $U(\ell) = \{v \in V \mid \mathrm{InU}(v) = 1\}$. We show that the following holds.

Lemma B.3. *If* $\mathrm{Ver}(G, I, v) = \text{accept}$ *for all* $v \in V$*, then:*

(1) The cluster graph $G'(r, U(\ell))$ is connected.

(2) For any 0-cluster C, we have that $|C| = r$.

(3) Let C, C' be 0-clusters that are connected by an edge in $G'(r, U(\ell))$, then $S'(C) = S'(C')$.

Proof. We show that indeed all the properties above hold.

(1) Since every node v accepts, it holds that $\min_{C \in \text{SeenClusters}(v)} \text{dist}_G(v, C) \leq r$. In particular $\text{dist}_G(v, V \setminus U(\ell)) \leq r$ for all $v \in V$, and hence by Lemma B.2 the graph $G'(r, U(\ell))$ is connected.

(2) Let $v \in C$, then $C \in \text{SeenClusters}(v)$ (otherwise v rejects). This implies that $|C| = r$, since v asserts that $|C| = r$ for every $C \in \text{SeenClusters}(v)$.

(3) Since C, C' are connected by an edge, then by definition $\text{dist}_G(C, C') \leq 2r + 1$, and by (2) we have $|C| = r$. Therefore, for a vertex $v \in C$ we have that $C, C' \subseteq B_{4r+1}(v)$. In particular, $C, C' \in \text{SeenClusters}(v)$ and v asserts that $S'(C) = S'(C')$.

The lemma implies that there exists some string S', such that for every 0-cluster C, $S'(C) = S'$. This follows from the cluster graph $G'(r, U(\ell))$ being connected, and that for every edge $\{C, C'\} \in E(G'(r, U(\ell)))$ we have that $S'(C) = S'(C')$. Moreover, since for any $v \in V$, we have $|\text{SeenClusters}(v)| \geq 1$ (otherwise v rejects), then node v outputs the string S', and soundness follows. □